A RAM-Efficient Implementation of Falcon

Thomas Pornin

NCC Group, thomas.pornin@nccgroup.com

7 September, 2026

Abstract. We present a RAM-efficient implementation of Falcon: RAM usage has shrunk to about 11 kB, down from about 31 kB in the previous implementation of Falcon-512. This code is furthermore faster on Arm Cortex M4, with average signature generation cost down to 13.45 million cycles. Optimization techniques include a novel variant of the FFT, replacement of some floating-point operations with modular integer computations, delayed addition of input within the Fast Fourier sampling process, and an alternate signature reassembly process. More than half of the floating-point operations have been removed. The resulting implementation is now small enough to allow use in small embedded systems such as smart cards.

1 Introduction

Falcon is a post-quantum signature scheme[24]; it was submitted to the NIST post-quantum cryptography standardization project[17] in 2017, and has been selected for becoming the NIST standard FIPS 206, under the name "FN-DSA". Falcon is a hash-and-sign scheme based on an NTRU lattice. In order to achieve small signatures and keys while maintaining the expected security level, a short vector is obtained with an appropriate randomization of Babai's Nearest Plane algorithm[2], as in Klein's sampler[16], except that an NP variant known as Fast Fourier Orthogonalization is employed, for much improved performance[10]. Falcon's parameter sets use a degree $n = 2^\ell$; the two main choices are $n = 512$ (for NIST's "level I" security, similar in strength to AES-128) and $n = 1024$ (for "level V", similar to AES-256).

The verification algorithm of Falcon is very fast, uses little RAM, and fits in a small code footprint. The key pair generation algorithm is complicated and large; several previous works have been optimizing that algorithm[20,21,23], which now runs with a large but tolerable CPU cost (e.g. it is still much faster than RSA key pair generation) and moderate RAM requirements (about 12 or 23 kB, depending on the degree n). The signature generation algorithm, though, still has significant RAM requirements (about 31 kB for $n = 512$, 61 kB for $n = 1024$), which make it difficult to use on the smallest embedded systems such as smart cards, where RAM is usually a very scarce resource. Moreover, the signature generation algorithm, as specified in [24], uses floating-point operations, which are expensive on microcontrollers with no hardware FPU (or whose FPU does not support the required precision), and whose masking against side-channel analysis attacks is expected to be doable but difficult[3,4,5,7,8,15]. This paper aims at reducing both the RAM requirements, and the number of floating-point operations, in Falcon's signature generation algorithm.

As will be detailed in the rest of the paper, the RAM requirements for signature generation have been slashed by a factor of 3, down to about 11 kB (for $n = 512$) and 21 kB (for

$n = 1024$). More than half of the floating-point operations have been removed, with the side-effect of improving performance on an Arm Cortex M4 microcontroller (cost has dropped to 13.4 million cycles at $n = 512$, down from 17.0 millions). On large CPUs (x86, aarch64) which have an efficient hardware FPU with SIMD instructions, this new implementation is moderately slower (cost is increased by 17% on a Skylake-class Intel x86, by 27% on a 64-bit Arm Cortex A76). This new implementation is available at:

`https://github.com/pornin/c-fn-dsa-alt`

FN-DSA Standard. As these lines are written, the NIST standard FIPS 206 is not published yet; a draft of that standard has been announced and has been sent into NIST's approval process at some point around mid-2025, and has since been stuck there for unclear reasons. There is no available estimate for when that draft of FIPS 206 will be made public, let alone the final FIPS 206. For the purposes of this paper, we use as reference the implementation available at:

`https://github.com/pornin/c-fn-dsa`

which matches, as of 30 July, 2026, my best guess of what the FIPS 206 draft will contain. This is based on what NIST announced in various places (workshops, mailing-lists...). Compared to Falcon as submitted, the main changes, apart from inconsequential endianness choices in encodings, are the following:

- The message to sign is hashed into a message representative μ along with some context information and a hash of the public key, as is done in ML-DSA[19]. Only μ is used afterwards.
- A new 40-byte random seed is generated for each signing attempt, instead of reusing the same seed (this improves provability of the security[11]).
- The infinity norm of a valid signature is enforced to be at most 840 (down from 2047).
- The public key is in NTT representation.

This implementation is what will be called in this paper the "standard" implementation. There is no guarantee that when the FIPS 206 draft is published, it will really match this implementation.

Interoperability and Compliance. A thorny issue is the question of what is actually a "compliant" Falcon/FN-DSA implementation. Right now, with no published standard nor even draft, there is no real notion of compliance, but NIST's plans seem to be, for the moment, to require strict reproducibility of test vectors. An implementation should provide a test API which receives a source random seed, and everything is deterministically specified from that seed. The point is to be able to validate that the Gaussian sampler, which is a rather complicated procedure, has been implemented properly and does not generate values with an exploitable bias; verifying strict reproducibility of test vectors goes a long way toward showing that an implementation at least follows the expected code structure. Unfortunately, this means that a truly compliant signature generator MUST perform exactly the same floating-point computations in the same order as the reference code, because any deviation in the last few bits of the inputs to the Gaussian sampler will at least occasionally induce divergences in the rejection sampling process.

The implementation we describe here is *safe* and *fully interoperable*, but would not be strictly compliant in the sense above:

- Safety: all operations are done with enough precision to ensure that the security analysis of Falcon still applies (in fact, this new implementation has somewhat *better* precision than the "standard" code, see section 5.3).
- Interoperability: this new implementation uses the same formats for public keys, private keys and signatures; signatures generated by the "standard" implementation are verifiable by our new code, and vice versa. *Full* interoperability denotes that this also applies to private keys: all private keys generated by one implementation are usable by the other (in fact the key pair generator code is completely unchanged).

Our new implementation will *most of the time* reproduce test vectors, by using the same private keys, input messages, and source random seeds. However, divergences will occur with probability about 1/8000. If NIST indeed insists on strict reproducibility, then our new code would not be formally compliant. As explained in section 5.4, we deliberately broke that almost-reproducibility, but it is easy to reactivate (a one-line code change).

Fixed-Point. A recently published implementation uses fixed-point operations instead of floating-point[1]. Their implementation is, like ours, safe and interoperable (it is not always *fully* interoperable because it enforces some extra requirements on private keys, so that it cannot use some of the private keys generated by the "standard" code). The fixed-point approach should promote performance on FPU-less hardware, and, more importantly, help with masking strategies against side-channel attacks. That fixed-point work is *a priori* complementary with the optimizations described here. A future work is to merge both implementations, i.e. make the code presented here use fixed-point operations instead of floating-point.

Main Ideas. The main optimization ideas that we use are the following:

- We design a new FFT variant, which removes about half of the floating-point operations, and furthermore allows halving the storage size of some polynomials. This is described in section 3. This new FFT might be applicable to other schemes than Falcon; it can also be turned into a new NTT that works with prime moduli equal to -1 modulo n, rather than the classic 1 modulo $2n$.
- We apply the short representation of polynomials to the intermediate node values in the Falcon tree. Moreover, the top-level node of the tree is built specially, on-the-fly, with most polynomials being computed over integers, using computations modulo $q = 12289$ and $q' = 18433$; our range analysis shows that the process is exact and cannot hit any integer overflow (see section 4).
- Only the fractional part of target vectors is used in sampling, and it is moreover added later in the process; this improves precision and avoids some FFT computations. Similarly, the final output polynomial in FFT representation is discarded (and actually not computed at all), since the signature is recomputed from the sampled integers; this removes the need for any inverse FFT in the whole scheme. Section 5 describes these optimizations.

2 Falcon Signature Generation

We recall in this section the Falcon signature generation scheme (and parts of the key pair generation and verification). Compared with the Falcon specification[24], we dispatch the operations over a slightly different structure of sub-algorithms, but the actual computations are not changed. We also skip the description of some operations, in particular the Gaussian sampling, that we do not modify.

2.1 Notations

Polynomial Ring. The main parameter of Falcon is the degree $n = 2^\ell$, for some integer $\ell \geq 1$. The standard Falcon degree is either $n = 512$ (for "level I" security) or 1024 (for "level V"). Weak versions, meant for research purposes and to help development, use lower degrees, from $n = 4$ to 256. We consider polynomials modulo the cyclotomic polynomial $\Phi_n = X^n + 1$. Since Φ_n is irreducible over $\mathbb{Q}$, the ring $\mathbb{Q}[X]/\Phi_n$ is a field. The same does not hold for $\mathbb{R}[X]/\Phi_n$ (for $n \geq 4$) nor for $\mathbb{C}[X]/\Phi_n$; in the latter, Φ_n splits, with roots being $\zeta_{n,j} = e^{i(2j+1)\pi/n}$ for integers $j = 0$ to $n - 1$.

A polynomial $a \in \mathbb{C}[X]/\Phi_n$ can be expressed in *coefficient representation*, i.e. as a sequence of n elements of $\mathbb{C}$ denoted $a[j]$ for $j = 0$ to $n - 1$, fulfilling $a = \sum_{j=0}^{n-1} a[j]X^j$. The *FFT representation* of a is the sequence of n values $a(\zeta_{n,j})$ for $j = 0$ to $n-1$. As will be detailed later, the FFT representation is redundant for the kind of polynomials used in Falcon, and we omit half or sometimes three quarters of it.

Quotient Ring. For $a, b \in \mathbb{Z}[X]/\Phi_n$ (polynomials with integral coefficients) and some integer $q > 1$, we can consider equality and operations modulo q: $a = b \bmod q$ if and only if there exists some $w \in \mathbb{Z}[X]/\Phi_n$ such that $a = b + qw$. Similarly, we say that a is invertible modulo q if and only if there exists some $t \in \mathbb{Z}[X]/\Phi_n$ such that $at = 1 \bmod q$. Note that this implies that $a \neq 0$. By extension, we say that a is invertible modulo 1 if and only if $a \neq 0$.

Equivalently, we can consider a polynomial "modulo q" by interpreting it as being defined over $\mathbb{Z}_q[X]/\Phi_n$, with $\mathbb{Z}_q$ being the ring of integers modulo q. If q is prime and is such that $q = 1 \bmod 2n$, then multiplications and divisions of polynomials in $\mathbb{Z}[X]/\Phi_n$ can be considerably sped up by applying the NTT. In Falcon, we mainly use $q = 12289$, which matches these conditions. In the rest of this paper, we will also use the alternate modulus $q' = 18433$, for which the NTT is also efficient.

Adjoint. We define the *(Hermitian) adjoint* of polynomial $a \in \mathbb{C}[X]/\Phi_n$ to be the polynomial a^* which is such that $a^*(\zeta) = \overline{a(\zeta)}$ for all roots ζ of Φ_n in $\mathbb{C}$. If the coefficients $a[j]$ are all real, then $a^* = a[0] - \sum_{j=1}^{n-1} a[n-j]X^j$. For such a real polynomial, we can also obtain a^* by composition: $a^* = a(1/X)$.

Like complex conjugation, the adjoint is compatible with additions and multiplications: for polynomials a and b, we have $(a + b)^* = a^* + b^*$, $(ab)^* = a^*b^*$, and $(a/b)^* = a^*/b^*$.

Inner Product and Norm. For polynomials $a, b \in \mathbb{R}[X]/\Phi_n$, their inner product is:

$$\langle a, b \rangle = \frac{1}{n} \sum_{\Phi_n(\zeta)=0} a(\zeta)\overline{b(\zeta)} = \sum_{j=0}^{n-1} a[j]\,b[j]$$

This inner product allows defining the norm of a polynomial a as $\|a\| = \sqrt{\langle a, a\rangle}$.

We also define the *infinity norm* of a polynomial a as $\|a\|_\infty = \max_{j=0}^{n-1} |a[j]|$.

The inner product and the norms extend to vectors of polynomials by computing them coefficient-wise; e.g. $\|(a, b)\| = \|a\| + \|b\|$, and $\|(a, b)\|_\infty = \max(\|a\|_\infty, \|b\|_\infty)$.

Splitting and Merging. A polynomial $a \in \mathbb{Q}[X]/\Phi_n$ can be split into even-indexed and odd-index coefficients:

$$a = a_{\text{even}}(X^2) + X a_{\text{odd}}(X^2)$$

with $a_{\text{even}} = \sum_{j=0}^{n/2-1} a[2j]X^j$ and $a_{\text{odd}} = \sum_{j=0}^{n/2-1} a[2j{+}1]X^j$; both a_{even} and a_{odd} are naturally in $\mathbb{Q}[X]/\Phi_{n/2}$. The split and merge operations denote splitting and its opposite: $\mathsf{split}(a) = (a_{\text{even}}, a_{\text{odd}})$, and $\mathsf{merge}(a_{\text{even}}, a_{\text{odd}}) = a$.

FFT. The *Fast Fourier Transform (FFT)* is an efficient way to compute, for a polynomial a, its FFT representation, which is the list of values $a(\zeta)$ for all roots ζ of Φ_n in $\mathbb{C}$. The gist of the FFT is related to splitting: if $(a_{\text{even}}, a_{\text{odd}}) = \mathsf{split}(a)$, then, for any root ζ of Φ_n, $-\zeta$ is also a root of Φ_n, and we have:

$$\begin{aligned} a(\zeta) &= a_{\text{even}}(\zeta^2) + \zeta a_{\text{odd}}(\zeta^2) \\ a(-\zeta) &= a_{\text{even}}(\zeta^2) - \zeta a_{\text{odd}}(\zeta^2) \end{aligned}$$

Moreover, ζ^2 is a root of $\Phi_{n/2}$; thus, given a polynomial a, we can split it into a_{even} and a_{odd}, recursively apply the FFT on both (in $\Phi_{n/2}$), and then inexpensively recombine them into the FFT representation of a. The FFT has an asymptotic cost of $O(n \log n)$ operations, which is why it is computationally attractive. We denote by FFT and invFFT the FFT and its inverse operation, respectively.

If $a \in \mathbb{R}[X]/\Phi_n$, then for every root ζ of Φ_n, its complex conjugate $\overline{\zeta}$ is also a root of Φ_n, and $a(\overline{\zeta}) = \overline{a(\zeta)}$ (the real coefficients of a are unchanged by complex conjugation); thus, we can omit half of the elements of the FFT representation of a, since they can always be recomputed by complex conjugation.

The FFT representation is compatible with operations on polynomials: addition, subtraction, multiplication and division of polynomials can be performed coefficient-wise on the FFT representations of the polynomials. Unfortunately, the ζ roots are complex numbers with irrational real and imaginary parts; this means that in practice, the FFT representation of a polynomial a must use an approximate representation, usually some floating-point or fixed-point format.

Vectors and Matrices. Some operations use vectors of polynomials, and matrices whose coefficients are polynomials. They will be denoted with a bold font, e.g. $\mathbf{t} = (t_0, t_1)$. Vectors use the row convention, i.e. the vector coordinates are shown horizontally; when applying a linear operation $\mathbf{B}$ to a vector, the matrix is on the right of the operand, e.g. $\mathbf{tB}$.

2.2 Key Pair Generation

Base Polynomials and the NTRU Equation. Key pair generation consists in sampling two polynomials f and g from $\mathbb{Z}[X]/\Phi_n$ with a given Gaussian distribution for each coefficient. The key pair generation algorithm is supposed to enforce the following rules:

- Coefficients of f and g have a limited range, since they are encoded in the private key over a limited number of bits: $\|f\|_\infty \leq \delta_n$ and $\|g\|_\infty \leq \delta_n$ for a value δ_n which depends on n: $\delta_{1024} = 16$, $\delta_{512} = \delta_{256} = 32$, $\delta_{128} = \delta_{64} = 64$, and $\delta_n = 128$ for $n \leq 32$.
- f is invertible modulo q.
- To allow generation of short signatures, some norms are limited:

$$\max\left(\|(g, -f)\|, \left\|\left(\frac{qf^*}{ff^* + gg^*}, \frac{qg^*}{ff^* + gg^*}\right)\right\|\right) \leq 1.17\sqrt{q}$$

- There exist polynomials $F, G \in \mathbb{Z}[X]/\Phi_n$ such that $fG - gF = q$ (this is called the *NTRU equation*). Moreover, $\|F\|_\infty \leq 127$ and $\|G\|_\infty \leq 127$. When there is a solution (F, G) to the NTRU equation, then there are an infinity of solutions (which are all the pairs $(F + kf, G + kg)$ for any $k \in \mathbb{Z}[X]/\Phi_n$), and in general several will match the condition on the infinity norms; the key pair generation algorithm chooses one solution, whose F is part of the encoding of the private key into bytes.

G is not part of the encoding of the private key, because it can be reconstructed from the NTRU equation: given f, g and F, we have $G = (q + gF)/f$. Since the division by f is exact over the integers, G can be evaluated modulo q, in which case the equation simplifies into $G = gF/f \bmod q$; the division can be done coefficient-wise in NTT representation modulo q. Since $\|G\|_\infty \leq 127 < q/2$, the result modulo q can be converted to plain integers, yielding the actual G.

There are several known methods to compute (F, G) from (f, g). The recommended method, which minimizes both RAM and CPU costs[20], requires that the resultants of f and g with Φ_n are odd, coprime integers. Any key pair generation implementation is free to reject some (f, g) pairs for which it cannot derive a solution (F, G) even if, mathematically, such a solution exists. When a candidate (f, g) pair is rejected, a new pair is sampled (*both* polynomials are resampled; the sampler for f and g is fully defined by NIST and internally uses SHAKE256).

f, g, F and G collectively make up the secret NTRU lattice basis:

$$\mathbf{B} = \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$$

The inverse of this matrix is:

$$\mathbf{B}^{-1} = \frac{1}{q}\begin{bmatrix} -F & f \\ -G & g \end{bmatrix}$$

The same lattice can be generated by the public matrix:

$$\mathbf{Q} = \begin{bmatrix} h & -1 \\ q & 0 \end{bmatrix}$$

with $h = g/f \bmod q$. The Falcon public key is an encoding of h.

Falcon Tree. The *Falcon tree* is the result of preprocessing a private key (f, g, F, G) into a number of polynomials which are nominally in $\mathbb{Q}[X]/\Phi_n$, but are usually expressed in FFT representation. These polynomials are organized in a binary tree structure:

- Inner nodes have a value, which is a polynomial, and two children (labelled left and right) which are themselves trees.
- Leaf nodes are real numbers.
- A tree is represented by its root node, which is either an inner node, or a leaf value.
- The full tree starts with a root whose polynomial value is in $\mathbb{Q}[X]/\Phi_n$; the two children of a tree node that contains a polynomial in $\mathbb{Q}[X]/\Phi_m$ themselves have values that are polynomials in $\mathbb{Q}[X]/\Phi_{m/2}$. The degree is thus halved at each level; when an inner node holds a value expressed in $\mathbb{Q}[X]/\Phi_2$, its two children are leaves, i.e. real numbers.

The algorithm FalconTreeTop (algorithm 1) computes the Falcon tree; it internally uses FalconTreeInner (algorithm 2), which is recursive.

Algorithm 1 FalconTreeTop

Input: $f, g, F, G \in \mathbb{Q}[X]/\Phi_n$
Output: Binary tree T

$a \leftarrow ff^* + gg^*$ ▷ $a \in \mathbb{Q}[X]/\Phi_n$ and $a^* = a$
$b \leftarrow Ff^* + Gg^*$
$c \leftarrow FF^* + GG^*$
$d \leftarrow c - bb^*/a$ ▷ $d \in \mathbb{Q}[X]/\Phi_n$ and $d^* = d$
$L \leftarrow b/a$
T.value $\leftarrow L$
T.left $\leftarrow$ FalconTreeInner(a)
T.right $\leftarrow$ FalconTreeInner(d)
return T

Algorithm 2 FalconTreeInner

Input: $a \in \mathbb{Q}[X]/\Phi_m$ such that $a = a^*$
Output: Binary tree T

$(b, c) \leftarrow$ split(a) ▷ $b, c \in \mathbb{Q}[X]/\Phi_{m/2}$; also: $b^* = b$
$L \leftarrow c^*/b$
$d \leftarrow b - cc^*/b$ ▷ $d^* = d$
T.value $\leftarrow L$
if $m \geq 8$ **then**
    T.left $\leftarrow$ FalconTreeInner(b)
    T.right $\leftarrow$ FalconTreeInner(d)
else
    T.left $\leftarrow b(0)$
    T.right $\leftarrow d(0)$
return T

The Falcon specification describes the tree building with a single function (called ffLDL*) which takes over the roles of both FalconTreeTop and FalconTreeInner. Here, we use two separate functions to highlight some properties that we will use later on, in particular that the input of FalconTreeInner is a single polynomial which is futhermore equal to its adjoint.

In the Falcon specification, once the tree has been computed, its leaves are normalized: if a leaf value is x, then it is replaced with $\sigma_n/\sqrt{x}$ for a constant σ_n defined as:

$$\sigma_n = 1.17\sqrt{\frac{q \log\left(4n\left(1 + 2^{32}\sqrt{\max(2, n/4)}\right)\right)}{2\pi^2}}$$

In the description shown here, the normalization step is performed right before using the leaf in a call to SamplerZ, even though it could be done as part of the tree building.

Computing the Falcon tree involves splitting polynomials, and computing some arithmetic operations on these polynomials. In plain coefficient representation, splitting a polynomial is trivial, but polynomial divisions are very much not so. It is possible to compute a division in $\mathbb{Q}[X]/\Phi_n$ by performing an extended Euclidean GCD between the divisor and Φ_n. Another method, which is applicable here thanks to the structure of Φ_n, is to perform a descent in a tower of fields: for a polynomial $b \in \mathbb{Q}[X]/\Phi_n$, we can define the Galois conjugate of b as $b^\times(X) = b(-X)$ (i.e. we simply negate the odd-indexed coefficients), and then compute $a/b = (ab^\times)/(bb^\times)$. The product $bb^\times$ happens to have only even-indexed coefficients (the odd-indexed coefficients are null), hence it can be expressed as $bb^\times = b'(X^2)$ for a polynomial $b' \in \mathbb{Q}[X]/\Phi_{n/2}$. Using this method recursively on b', we finally transform the original fraction a/b into c/r with r being a constant polynomial; at that point, the division can be applied on the coefficients of c one by one. As detailed in [20], the value r is really the resultant of b with Φ_n.

Regardless of the division method used, the resulting polynomial coefficients will have very large denominators, in the thousands of bits, implying large RAM usage and poor performance. To make the tree building tolerably efficient, the Falcon specification uses the FFT: the source polynomials f, g, F and G are converted to the FFT representation (as $n/2$ complex values each), and all operations are performed in the FFT domain. Addition, subtraction, multiplication and division of polynomials are then done coefficient-wise. Splitting in the FFT domain is denoted splitfft in the Falcon specification, and can be done efficiently: for any root ζ of Φ_n, ζ^2 is a root of $\Phi_{n/2}$, and:

$$\begin{aligned} a_{\text{even}}(\zeta^2) &= (a(\zeta) + a(-\zeta))/2 \\ a_{\text{odd}}(\zeta^2) &= (a(\zeta) - a(-\zeta))/(2\zeta) \end{aligned}$$

The Falcon tree depends only on the private key (the f, g, F and G polynomials) and, as such, can be precomputed, though this is not mandatory; it is also possible to recompute it at signing time. Signature generation involves a depth-first walk of the tree in right-to-left order; this can be combined with a dynamic generation that only needs to temporarily store the polynomials in the path from the root to the current leaf, thus not needing the RAM space that would be required for storing the whole tree.

2.3 Signature Generation

Signature of a message M first requires pre-processing M along with some contextual information (e.g. if M is really a hashed value, then an identifier for the used hash function is included) and a hash of the public key; the output is a 64-byte string μ. Then, a random 40-byte seed r is generated, and hashed along with μ to feed a rejection sampling process that outputs a polynomial $c \in \mathbb{Z}_q[X]/\Phi_n$. The 40-byte seed will be included in the signature, so that verifiers may recompute c. If the signature generation fails (it can happen, with a relatively low probability), a new 40-byte seed is generated[1], but μ is reused. We omit here the details of the generation of c.

We suppose for now that the private key was decoded into f, g and F, then G was recomputed using the NTRU equation modulo q, and the Falcon tree T was built. Given c, the signature generation proceeds as follows:

1. Compute $\mathbf{t} = (c, 0)\mathbf{B}^{-1} = (-cF/q, cf/q)$.
2. Sample a vector $\mathbf{z}$ of integral polynomials through the ffSampling algorithm.
3. Compute $\mathbf{s} = (\mathbf{t} - \mathbf{z})\mathbf{B}$.
4. If $\|\mathbf{s}\|$ and $\|\mathbf{s}\|_\infty$ are low enough, then try to encode the second element of $\mathbf{s}$ into bytes; if the encoding succeeds, then this is the signature. Otherwise, the process starts again with a new random 40-byte seed r and thus a new polynomial c.

The intuition behind this process is the following: the signature is a vector $\mathbf{e}$ which is part of the lattice defined by the public basis $\mathbf{Q}$ (for which $\mathbf{B}$ is also a basis) and which is *close* to the target vector $(c, 0)$. The lattice consists of all vectors $\mathbf{kB}$ for any integral polynomial $\mathbf{k}$; the vector $\mathbf{t}$ is such that $\mathbf{tB}$ is *equal* to $(c, 0)$, but $\mathbf{t}$ is not integral ($\mathbf{B}^{-1}$ starts with a $1/q$ factor). The ffSampling procedure finds a vector $\mathbf{z}$ which is integral such that $\mathbf{zB}$ is close to $\mathbf{tB}$, at which point we can set $\mathbf{e} = \mathbf{zB}$. The signature itself is then an encoding of $\mathbf{s} = (c, 0) - \mathbf{e} = (\mathbf{t} - \mathbf{z})\mathbf{B}$, which has low norm. For appropriate security, $\mathbf{e}$ must not be *too close* to the target $(c, 0)$, since that would otherwise leak too much information about the secret basis $\mathbf{B}$; the ffSampling procedure is a variant of Klein's sampler[16], which is itself a randomized variant of Babai's Nearest Plane algorithm[2]. Klein's sampler implementation has a large cost (in $O(n^2)$); ffSampling organizes it with a recursive splitting process which benefits from the FFT representation, for a $O(n \log n)$ cost[10].

ffSampling (algorithm 3) uses the Falcon tree; namely, it receives as parameters a degree m, a target vector $\mathbf{t}$ of polynomials in $\mathbb{Q}[X]/\Phi_m$, and a Falcon tree whose root node holds a value which is itself an element of $\mathbb{Q}[X]/\Phi_m$. The process is recursive, the degree m being halved whenever going one level deeper in the recursion; when $m = 1$, the tree is reduced to a single real value, which is used as one of the parameters of SamplerZ.

The SamplerZ function takes as inputs a target real number x and a width σ', and it returns an integer z sampled from a Gaussian distribution centred on x and with standard deviation σ'. SamplerZ internally consumes random bits from a random source. We will not detail here how SamplerZ works, except to express an important characteristic, which is that if $\mathsf{SamplerZ}(x, \sigma')$ returns the integer z for a given state of its internal random source, then, for any integer j, $\mathsf{SamplerZ}(x+j, \sigma')$ should return $z+j$ for the same internal random source

[1] This is one of the differences between the original Falcon[24] and the presumed future FIPS 206; in the original Falcon, r was reused in that case.

Algorithm 3 ffSampling

Input: degree m, target $\mathbf{t} = (t_0, t_1) \in (\mathbb{Q}[X]/\Phi_m)^2$, a Falcon tree T whose root has degree m
Output: $\mathbf{z} = (z_0, z_1) \in (\mathbb{Z}[X]/\Phi_m)^2$

if $m = 1$ **then**
  $\sigma' \leftarrow \sigma_n / \mathsf{T}$ ▷ At degree 1, the tree is a single real number.
  $z_0 \leftarrow \mathsf{SamplerZ}(t_0, \sigma')$
  $z_1 \leftarrow \mathsf{SamplerZ}(t_1, \sigma')$
  return (z_0, z_1)
$\mathbf{v} \leftarrow \mathsf{split}(t_1)$
$\mathbf{y} \leftarrow \mathsf{ffSampling}(m/2, \mathbf{v}, \mathsf{T.right})$
$z_1 \leftarrow \mathsf{merge}(\mathbf{y})$
$L \leftarrow \mathsf{T.value}$
$t'_0 \leftarrow t_0 + (t_1 - z_1)L$
$\mathbf{u} \leftarrow \mathsf{split}(t'_0)$
$\mathbf{w} \leftarrow \mathsf{ffSampling}(m/2, \mathbf{u}, \mathsf{T.left})$
$z_0 \leftarrow \mathsf{merge}(\mathbf{w})$
return (z_0, z_1)

state, and consume exactly as many bits from that source. In other words, SamplerZ first splits the input x into an integer x_i and a fractional part x_f such that $x = x_i + x_f$ and $0 \leq x_f < 1$; it then performs the sampling using x_f alone, and finally adds back x_i to the sampled integer.

2.4 Signature Verification

Signature verification works over the message M, the public lattice basis $\mathbf{Q}$, and the signature, which nominally includes r and the vector $\mathbf{s} = (s_0, s_1)$. The verifier computes the message representative μ, then (using r) the target polynomial c. The signature is deemed valid if all of the following hold:

$$\begin{aligned} c &= s_0 + s_1 h \mod q \\ \|\mathbf{s}\| &\leq \beta \\ \|\mathbf{s}\|_\infty &\leq \gamma_\infty \end{aligned}$$

The thresholds for the last two conditions are scheme parameters: $\beta = 1.1\sigma_n\sqrt{2n}$, and $\gamma_\infty = 840$. The first condition really expresses that $(c, 0) - \mathbf{s}$ is part of the lattice generated by the public matrix $\mathbf{Q}$. Since this equation allows recomputing s_0 from s_1 (using $s_0 = c - s_1 h \mod q$), only s_1 and the random seed r need to be actually encoded in the signature. As for the public matrix $\mathbf{Q}$, its contents can be conveyed by encoding only the public polynomial h, whose coefficients are all integers in the $[0, q-1]$ range.

3 Optimizing the FFT

In the Falcon specification, the FFT is used throughout the process in order to speed up operations and in particular polynomial divisions. Polynomials f, g, F, G and c are all converted to FFT representation; the computations of $\mathbf{t} = (-cF/q, cf/q)$ and all split, merge and arithmetic operations on polynomials in ffSampling are then performed on FFT representations. At the deepest recursion level, polynomials t_0 and t_1 are modulo $\Phi_1 = X + 1$, i.e. they have degree zero, and are equal to their FFT representation, so that their values can be used directly in SamplerZ. Finally, the output of ffSampling can be used to compute $\mathbf{s}$, still working with FFT representations, and only at the end is $\mathbf{s}$ reconverted back to coefficient representation through the inverse FFT. In total, following the Falcon specification, five FFTs and two inverse FFTs are performed on polynomials. Making these operations more efficient thus seems important for performance.

Here, we primarily target small embedded systems, such as microcontrollers using an Arm Cortex M4 core. Such systems do not offer hardware support for floating-point operations with the precision needed by Falcon; these operations must then be emulated with integer code, which should furthermore avoid leaking secret information through side-channels, and thus should at least operate in a constant-time way. The cost of floating-point operations thus dominates that of the FFT, and the primary goal is to reduce the number of such operations. Our assembly-optimized implementations have the following costs:

- Addition: 72 cycles
- Combined addition-subtraction: 87 cycles
- Multiplication: 36 cycles

A combined addition-subtraction is obtaining both $x + y$ and $x - y$ for given inputs x and y. Addition is counter-intuitively more expensive than multiplication because it involves shifting operands to make them share the same exponent, and that shift is expensive, especially since the shift amount may or may not exceed the size of a register, and constant-time discipline forces us to always cover all cases. Negation, and scaling by a power of two (in particular, dividing a value by 2), are inexpensive, so that the cost of a subtraction can be considered to be the same as that of an addition (in all of the subsequent text, we will loosely talk of "additions" to designate both additions and subtractions, since they have the same cost). These figures are in the context of our Falcon implementation, which uses the IEEE 754 "binary64" format, and for which it can be proven that NaNs, infinites and denormals never occur, neither as source operands nor as outputs; the only special values are zeros.

We will here estimate costs in numbers of additions (A) (which can be subtractions in practice), combined addition-subtractions (AS) and multiplications (M).

3.1 Classic FFT

We call "classic FFT" the algorithm that applies the "Cooley-Tukey butterfly" repeatedly[9][2], which really is about computing $a(\zeta)$ and $a(-\zeta)$, for roots ζ of Φ_m, from the values $a_{\text{even}}(\zeta^2)$ and $a_{\text{odd}}(\zeta^2)$, which are the FFT representations of a_{even} and a_{odd} in the ring $\mathbb{R}[X]/\Phi_{n/2}$.

[2] The algorithm was actually invented by Gauss more than a century before[12].

The corresponding inverse FFT simply reverts all operations using the reverse of the Cooley-Tukey butterfly, usually known as the "Gentleman-Sande butterfly"[13]. Practical implementations work in-place, to avoid extra memory allocation and unneeded data movement, which has the side-effect of returning the FFT representation in so-called "bit-reversal order". We define the rev function:

$$\mathsf{rev}\left(\alpha, \sum_{j=0}^{\alpha-1} x_j 2^j\right) = \sum_{j=0}^{\alpha-1} x_j 2^{\alpha-1-j}$$

with values $x_j \in [0, 1]$ (i.e. the x_j values are the digits of the representation of an integer from $[0, 2^\alpha - 1]$ in radix two, and the rev function reverses the order of that bit sequence). The classic FFT and its inverse are shown in algorithms FFT-Classic (algorithm 4) and invFFT-classic (algorithm 5).

Algorithm 4 FFT-classic

Input: $a = \sum_{j=0}^{n-1} a[j]X^j \in \mathbb{R}[X]/\Phi_n$, for $n = 2^\ell$
Output: FFT representation of a in bit-reversal order

for $\alpha = 1$ **to** $\ell - 1$ **do**
  $t \leftarrow n/2^{\alpha+1}$
  $m \leftarrow 2^\alpha$
  for $j = 0$ **to** $m/2 - 1$ **do**
    $s_r \leftarrow Z_r[m + j]$ ▷ $Z_r[\mathrm{rev}(\ell, j)] = \cos(\pi j/n)$
    $s_i \leftarrow Z_i[m + j]$ ▷ $Z_i[\mathrm{rev}(\ell, j)] = \sin(\pi j/n)$
    for $k = 0$ **to** $t/2 - 1$ **do**
      $k_0 \leftarrow 2^{\ell-\alpha} j + k$
      $k_1 \leftarrow k_0 + t$
      $(x_r, x_i) \leftarrow (a[k_0], a[k_0 + n/2])$
      $(y_r, y_i) \leftarrow (a[k_1], a[k_1 + n/2])$
      $z_r \leftarrow y_r s_r - y_i s_i$
      $z_i \leftarrow y_r s_i + y_i s_r$
      $(a[j_0], a[j_0 + n/2]) \leftarrow (x_r + z_r, x_i + z_i)$
      $(a[j_1], a[j_1 + n/2]) \leftarrow (x_r - z_r, x_i - z_i)$

The following points are noteworthy:

- Since the source polynomials are from $\mathbb{R}[X]/\Phi_n$, the FFT representation only returns $n/2$ complex values (the other $n/2$ values are implicit since they can be obtained through conjugation). FFT-classic returns $a(\zeta_{n,2j})$ for $j = 0$ to $n/2 - 1$, with $\zeta_{n,2j} = e^{i(4j+1)\pi/n}$.
- The FFT coefficients are in bit-reversal order, with the real and imaginary parts separate; namely:

  $$a(\zeta_{n,\mathrm{rev}(\ell,j)}) = \hat{a}[j] + i\hat{a}[j + n/2]$$

  for $j = 0$ to $n/2 - 1$, and $\hat{a}$ denoting the contents of the a array when FFT-classic terminates. This separation of the real and imaginery parts implies that the first formal iteration of the FFT algorithm does nothing (which is why the counter α starts at 1 in algorithm 4, instead of 0); it also promotes use of SIMD instruction sets since memory slots at successive addresses are processed identically.

Algorithm 5 invFFT-classic

Input: FFT representation of a in bit-reversal order
Input: $a = \sum_{j=0}^{n-1} a[j]X^j \in \mathbb{R}[X]/\Phi_n$

for $\alpha = \ell - 1$ **down to** 1 **do**
  $t \leftarrow 2^{\alpha-1}$
  $m \leftarrow n/2^\alpha$
  for $j = 0$ **to** $m/2 - 1$ **do**
    $s_r \leftarrow Z_r[m + j]$ ▷ Same $Z_r[]$ array as in FFT-classic.
    $s_i \leftarrow -Z_i[m + j]$ ▷ Same $Z_i[]$ array as in FFT-classic.
    for $k = 0$ **to** $t - 1$ **do**
      $k_0 \leftarrow 2^\alpha j + k$
      $k_1 \leftarrow k_0 + t$
      $(x_r, x_i) \leftarrow (a[k_0], a[k_0 + n/2])$
      $(y_r, y_i) \leftarrow (a[k_1], a[k_1 + n/2])$
      $(a[j_0], a[j_0 + n/2]) \leftarrow ((x_r + y_r)/2, (x_i + y_i)/2)$
      $(z_r, z_i) \leftarrow ((x_r - y_r)/2, (x_i - y_i)/2)$
      $a[k_1] \leftarrow z_r s_r - z_i s_i$
      $a[k_1 + n/2] \leftarrow z_r s_i + z_i s_r$

– Constants $Z_r[]$ and $Z_i[]$ take up a total of 16384 bytes of code space to support degrees up to $n = 1024$ and using 64-bit floating-point values; however, this size can easily be reduced to 4096 bytes by using the facts that $\sin(x) = \sin(\pi - x)$ and $\cos(x) = \sin(\pi/2 - x)$, so that only the values of $\sin(\pi j/n)$ for $j = 0$ to $n/2 - 1$ really need to be stored.
– The description of FFT-classic details the operations on real and imaginary parts separately, and not using an abstract notation with complex numbers, to highlight the cost of the multiplication by ζ (which is $s_r + is_i$ in the algorithm): such an operation requires four floating-point multiplications and two floating-point additions[3]. Our main optimization technique aims at avoiding multiplications of complex values.

Algorithm FFT-classic involves $\ell - 1$ iterations of the outer loop; each iteration involves combining the n elements by pairs, each combination involving two additions, two addition-subtractions, and four multiplications. In the first iteration, some marginal savings are possible: $m = 2$, so that $s_r = Z_r[2] = \cos(\pi/4) = \sin(\pi/4) = Z_i[2] = s_i$; thus, in that first outer iteration, $y_r s_r = y_r s_i$, $y_i s_i = y_i s_r$, and each pair may use three addition-subtractions and two multiplications. The total cost of FFT-classic is then $(n/2)((2\ell-4)\text{A}+(2\ell-1)\text{AS}+(4\ell-6)\text{M})$. Analysis for invFFT-classic is similar and yields the same cost estimate.

3.2 FFT of a Self-Adjoint Polynomial

Let $a \in \mathbb{R}[X]/\Phi_n$. We define that:

– a is *self-adjoint* if and only if: $a^* = a$
– a is *X-adjoint* if and only if: $a^* = Xa$

[3] This could also be implemented with three multiplications and five additions, with $(y_r + y_i)(s_r + s_i) = y_r s_r + y_i s_i + (y_r s_i + y_i s_r)$, and computing $y_r s_r$ and $y_i s_i$ separately; however, this would be more expensive since floating-point additions are more expensive than floating-point multiplications.

– a is $1/X$-*adjoint* if and only if: $a^* = (1/X)a$
– a is *neg-adjoint* if and only if: $a^* = -a$

The following remark shows that for all four categories, the full representation is redundant, and half of the coefficients can be inferred from the other half:

Remark 1. For $a \in \mathbb{R}[X]/\Phi_n$:

– If a is self-adjoint then $a[n/2] = 0$, and $a[n/2 + j] = -a[n/2 - j]$ for $j = 1$ to $n/2 - 1$.
– If a is X-adjoint then $a[j] = -a[n - 1 - j]$ for $j = 0$ to $n/2 - 1$.
– If a is $1/X$-adjoint then $a[1] = a[0]$, and $a[n + 1 - j] = -a[j]$ for $j = 2$ to $n/2$.
– If a is neg-adjoint then $a[0] = 0$, and $a[n/2 + j] = a[n/2 - j]$ for $j = 1$ to $n/2 - 1$.

As previously defined, for any polynomial $a \in \mathbb{R}[X]/\Phi_n$, $a^*(\zeta) = \overline{a(\zeta)}$ for all roots ζ of Φ_n. Apart from giving an easy way to transform a polynomial into its adjoint in FFT representation (just negate the imaginary part of each FFT coefficient), this implies that if a is self-adjoint, then all its FFT coefficients are real. In particular, in the representation used by FFT-classic, elements of index $n/2$ to $n - 1$ are the imaginary parts of the FFT coefficients, and thus they are all equal to zero in a self-adjoint polynomial, and can be omitted. Moreover, for both FFT-classic and invFFT-classic, values from the first half of the array ($a[0]$ to $a[n/2 - 1]$) are never combined with values from the second half ($a[n/2]$ to $a[n - 1]$). This allows making variants FFT-selfadj-classic and invFFT-selfadj-classic for self-adjoint polynomials: they work with half-size arrays ($n/2$ elements instead of n) and have half the cost of the full FFT-classic and invFFT-classic, i.e. $(n/4)((2\ell - 4)\text{A} + (2\ell - 1)\text{AS} + (4\ell - 6)\text{M})$.

We can do better. Some easily proven properties are the following:

Remark 2. Let $a \in \mathbb{R}[X]/\Phi_n$ a self-adjoint polynomial. If $(a_{\text{even}}, a_{\text{odd}}) = \text{split}(a)$, then a_{even} is self-adjoint, and a_{odd} is X-adjoint.

Remark 3. In $\mathbb{R}[X]/\Phi_n$, $1 + X$ is $1/X$-adjoint, and $X^{n/2}$ is neg-adjoint.

Let's consider $a \in \mathbb{R}[X]/\Phi_n$ a self-adjoint polynomial, and $(a_{\text{even}}, a_{\text{odd}}) = \text{split}(a)$. Define $b_{\text{odd}} = (1 + X)a_{\text{odd}}$ (this is multiplication in $\mathbb{R}[X]/\Phi_{n/2}$, since the output of split consists of half-degree polynomials). Then:

$$b_{\text{odd}}^* = (1 + X)^* a_{\text{odd}}^* = (1/X)X(1 + X)a_{\text{odd}} = b_{\text{odd}}$$

Thus, b_{odd} is self-adjoint. When applying polynomial a to the roots ζ of Φ_n, we obtain:

$$\begin{aligned} a(\zeta) &= a_{\text{even}}(\zeta^2) + \zeta a_{\text{odd}}(\zeta^2) \\ &= a_{\text{even}}(\zeta^2) + \frac{\zeta}{1 + \zeta^2} b_{\text{odd}}(\zeta^2) \end{aligned}$$

Note that a, a_{even} and b_{odd} are all self-adjoint, and thus $a(\zeta)$, $a_{\text{even}}(\zeta^2)$ and $b_{\text{odd}}(\zeta^2)$ are real numbers. It follows that $\zeta/(1 + \zeta^2) \in \mathbb{R}$, and, crucially, the multiplication of that value with $b_{\text{odd}}(\zeta^2)$ is between two reals, not two complex values, thus involving a single floating-point multiplication and no addition at all. Indeed, we have:

$$\frac{\zeta_{n,j}}{1 + \zeta_{n,j}^2} = \frac{e^{i(2j+1)\pi/n}}{1 + e^{i(2(2j+1))\pi/n}} = \frac{1}{e^{-i(2j+1)\pi/n} + e^{i(2j+1)\pi/n}} = \frac{1}{2\cos((2j + 1)\pi/n)}$$

This yields an FFT algorithm for self-adjoint polynomials which works in two steps:

1. Split the source polynomial into even-indexed and odd-indexed coefficients, then multiply by $1+X$ the odd-indexed half, and apply the same process recursively on both halves.
2. Recombine the polynomials by pairs, starting with degree 1 and upward, using the equations above and the combination values $\zeta/(1+\zeta^2)$.

Algorithm FFT-selfadj-new (algorithm 6) implements this algorithm. The constants $R[j]$ are such that:

$$R[2^{\alpha-2} + \mathsf{rev}(\alpha-2, j)] = \frac{1}{2\cos((4j+1)\pi/2^{\alpha})}$$

for all $j = 0$ to $2^{\alpha-2}-1$. $R[0]$ is not used, and $R[1] = \sqrt{2}/2$. To support polynomials of degree $n = 2^{\ell}$, the array $R[]$ must be computed for up to $\alpha = \ell$, which means a total of $2^{\ell-1} = n/2$ elements. For Falcon, we need to support up to $n = 1024$ and we use 64-bit floating-point values, so that the footprint of the constant $R[]$ array is 4096 bytes.

Algorithm 6 FFT-selfadj-new

Input: $a[0]$ to $a[n/2-1]$ for $a \in \mathbb{R}[X]/\Phi_n$ such that $n = 2^{\ell} \geq 4$ and $a^* = a$
Output: FFT representation of a in bit-reversal order (half-size)

1: **for** $\alpha = 0$ **to** $\ell - 3$ **do** ▷ Loop skipped if $\ell < 3$.
2: **for** $k = 2^{\ell-alpha-2} - 1$ **down to** 1 **do**
3: $k_0 \leftarrow 2^{\alpha+1}k$
4: **for** $j = 0$ **to** $2^{\alpha} - 1$ **do**
5: $a[k_0 + j + 2^{\alpha}] \leftarrow a[k_0 + j + 2^{\alpha}] + a[k_0 + j - 2^{\alpha}]$
6: **for** $j = 0$ **to** $2^{\alpha} - 1$ **do**
7: $a[j + 2^{\alpha}] \leftarrow 2a[j + 2^{\alpha}]$
8: **for** $j = 0$ **to** $n/4 - 1$ **do**
9: $(x, y) \leftarrow (a[j], a[j + n/4])$ ▷ If a is integral, values are converted to floating-point here.
10: $z \leftarrow y\sqrt{2}$ ▷ $\sqrt{2} = 2R[1]$
11: $(a[j], a[j + n/4]) \leftarrow (x + z, x - z)$
12: **for** $\alpha = 1$ **to** $\ell - 2$ **do** ▷ Loop skipped if $\ell < 3$.
13: $t \leftarrow 2^{\ell-\alpha-2}$
14: $m \leftarrow 2^{\alpha}$
15: **for** $j = 0$ **to** $m - 1$ **do**
16: $s \leftarrow R[m + j]$ ▷ $R[m + j] = 1/(2\cos((4\mathsf{rev}(\alpha, j) + 1)\pi/2^{\alpha+2}))$
17: **for** $k = 0$ to $t - 1$ **do**
18: $k_0 \leftarrow 2^{\ell-\alpha-1}j + k$
19: $k_1 \leftarrow k_0 + t$
20: $(x, y) \leftarrow (a[k_0], a[k_1])$
21: $z \leftarrow ys$
22: $(a[k_0], a[k_1]) \leftarrow (x + z, x - z)$

In FFT-selfadj-new, the first loop (lines 1-8) implements the splits and multiplications by $1+X$, and the third loop (lines 13-24) performs the recombinations; however, the first loop lacks its last iteration (last value of α is $\ell-3$, not $\ell-2$), and the third loop lacks its first iteration (first value of α is 1, not 0): these missing steps have been merged into the second loop (lines 9-12). Indeed, the last (missing) iteration of the first loop would just have multiplied half of

the value by 2, while the first (missing) iteration of the third loop would have multiplied the same values by $R[1] = \sqrt{2}/2$; the two operations can be done with a single multiplication by $\sqrt{2}$. The main reason for this merge is to highlight an important characteristic: if the source polynomial has integer coefficients (i.e. $a \in \mathbb{Z}[X]/\Phi_n$), then all operations in the first loop can be performed over plain integers, which is vastly faster than using floating-point on our target hardware. As we will see, this is the case in Falcon.

The first loop has $\ell - 2$ outer iterations, each involving $n/4$ additions, though some of these additions are really doublings; if the source polynomial is integral, then all these operations are on integers. The second loop uses $n/4$ addition-subtractions and $n/4$ multiplications. The third loop has $\ell - 2$ outer iterations, each using $n/4$ addition-subtractions and $n/4$ multiplications. The total cost of FFT-selfadj-new is then $(n/4)((\ell-2)\text{A} + (\ell-1)\text{AS} + (\ell-1)\text{M})$; if the input is integral, the $(\ell-2)\text{A}$ term disappears (it is replaced with integer additions). This cost is less than half the cost of FFT-selfadj-classic.

Algorithm invFFT-selfadj-new (algorithm 7) is a straightforward reversal of the operations of FFT-selfadj-new. It uses the constants $R'[]$ which are such that $R'[j] = 1/(2R[j])$ for all j (the size of R' is 4096 bytes when using 64-bit floating-point values and supporting degrees up to $n = 1024$). When the polynomial is known to be integral, floating-point values can be converted back to integers (with round-to-nearest) in the middle loop, and the third loop (the reverse of the first loop in FFT-selfadj-new) uses only operations on integers.

Algorithm 7 invFFT-selfadj-new

Input: FFT representation of a in bit-reversal order (half-size)
Output: $a[0]$ to $a[n/2-1]$ for $a \in \mathbb{R}[X]/\Phi_n$ such that $n = 2^\ell \geq 4$ and $a^* = a$

for $\alpha = \ell - 2$ **down to** 1 **do** ▷ Loop skipped if $\ell < 3$.
  $t \leftarrow 2^{\ell-\alpha-2}$
  $m \leftarrow 2^\alpha$
  for $j = 0$ **to** $m - 1$ **do**
    $s \leftarrow R'[m+j]$ ▷ $R'[m+j] = \cos((4\text{rev}(\alpha, j) + 1)\pi/2^{\alpha+2}) = 1/(2R[m+j])$
    for $k = 0$ to $t - 1$ **do**
      $k_0 \leftarrow 2^{\ell-\alpha-1}j + k$
      $k_1 \leftarrow k_0 + t$
      $(x, y) \leftarrow (a[k_0], a[k_1])$
      $(u, v) \leftarrow (x + y, x - y)$
      $(a[k_0], a[k_1]) \leftarrow (u/2, vs)$
for $j = 0$ **to** $n/4 - 1$ **do**
  $(x, y) \leftarrow (a[j], a[j + n/4])$
  $(u, v) \leftarrow (x + y, x - y)$
  $(a[j], a[j + n/4]) \leftarrow (u/2, v(1/\sqrt{8})$ ▷ If a is integral, values are converted to integers here.
for $\alpha = \ell - 3$ **down to** 0 **do** ▷ Loop skipped if $\ell < 3$.
  for $j = 0$ **to** $2^\alpha - 1$ **do**
    $a[j + 2^\alpha] \leftarrow a[j + 2^\alpha]/2$
  for $k = 1$ **to** $2^{\ell-alpha-2} - 1$ **do**
    $k_0 \leftarrow 2^{\alpha+1}k$
    for $j = 0$ **to** $2^\alpha - 1$ **do**
      $a[k_0 + j + 2^\alpha] \leftarrow a[k_0 + j + 2^\alpha] - a[k_0 + j - 2^\alpha]$

3.3 Improved FFT

Algorithm FFT-selfadj-new is an improved version of the computation of the FFT for self-adjoint polynomials. Here, we show how to also improve the general case, not just self-adjoint polynomials. The main technique is to decompose the source polynomial into a sum of two polynomials which can both be represented by self-adjoint polynomials: given $c \in \mathbb{R}[X]/\Phi_n$, we find self-adjoint polynomials a and b such that $c = a + X^{n/2}b$. We can then apply FFT-selfadj-new on both a and b. Using $\hat{a}$, $\hat{b}$ and $\hat{c}$ to denote the FFT representations of a, b and c, we have:

$$\hat{c}(\zeta) = \hat{a}(\zeta) + \zeta^{n/2}\hat{b}(\zeta)$$

The FFT representation returned by FFT-classic keeps only $a(\zeta_{n,2j})$ for $j = 0$ to $n/2 - 1$ (specifically, it returns only the first $n/2$ values of $a(\zeta_{n,\mathsf{rev}(\ell,j')})$ for increasing values of j', and the bit-reversal implies that all these $\mathsf{rev}(\ell, j')$ are even integers). We have $\zeta_{n,2j}^{n/2} = e^{i(4j+1)\pi/2} = i$ for all $j = 0$ to $n/2$, thus $\hat{c}(\zeta) = \hat{a}(\zeta) + i\hat{b}(\zeta)$ for all the ζ we are interested in. Since $a(\zeta)$ and $b(\zeta)$ are all real (a and b are self-adjoint), this implies that the FFT representations of a and b are really the real and imaginary parts of the elements of the FFT representation of c. In our chosen memory layout with real and imaginary parts separated by $n/2$ elements, $\hat{c}$ is then simply the concatenation of $\hat{a}$ and $\hat{b}$.

Note that $X^{n/2}b$ is neg-adjoint; we can thus easily obtain a and b:

$$a = (c^* + c)/2$$
$$b = (c^* - c)X^{n/2}/2$$

Algorithm FFT-new (algorithm 8) implements this method; the reverse operation is performed by invFFT-new (algorithm 9).

Algorithm 8 FFT-new

Input: $c \in \mathbb{R}[X]/\Phi_n$ such that $n = 2^\ell \geq 4$
Output: FFT representation of c in bit-reversal order

1: $c[0] \leftarrow 2c[0]$
2: $c[n/2] \leftarrow 2c[n/2]$
3: **for** $j = 1$ **to** $n/2 - 1$ **do**
4: $\quad (x, y) \leftarrow (c[j], c[n-j])$
5: $\quad (c[j], c[n-j]) \leftarrow (x - y, x + y)$ ▷ $c[0..n/2-1] = 2a$, and $c[n/2..n-1] = 2b$.
6: FFT-selfadj-new($c[0..n/2-1]$)
7: FFT-selfadj-new($c[n/2..n-1]$)
8: **for** $j = 0$ to $n - 1$ **do**
9: $\quad c[j] \leftarrow c[j]/2$

In these implementations, we apply a factor of 2 to a and b, so that the initial loop decomposes c into $2a$ and $2b$: this is done so that when c is an integral polynomial (as is the case in Falcon) then the decomposition can be performed with only integer operations. The corrective factor $1/2$ is applied on the FFT representation (halving is inexpensive on floating-point values).

Algorithm 9 invFFT-new

Input: FFT representation of c in bit-reversal order
Output: $c \in \mathbb{R}[X]/\Phi_n$ such that $n = 2^\ell \geq 4$

1: **for** $j = 0$ to $n - 1$ **do**
2: $\quad c[j] \leftarrow 2c[j]$
3: invFFT-selfadj-new($c[0..n/2 - 1]$)
4: invFFT-selfadj-new($c[n/2..n - 1]$)
5: $c[0] \leftarrow c[0]/2$
6: $c[n/2] \leftarrow c[n/2]/2$
7: **for** $j = 1$ **to** $n/2 - 1$ **do**
8: $\quad (x, y) \leftarrow (c[j], c[n - j])$
9: $\quad (c[j], c[n - j]) \leftarrow ((y + x)/2, (y - x)/2)$

Total cost of FFT-new is twice that of FFT-selfadj-new, plus an extra $n/2$ addition-subtractions when the the source polynomial is not integral, hence $(n/2)((\ell-2)\text{A}+\ell\text{AS}+(\ell-1)\text{M})$ in the general case, which is less than half the cost of FFT-classic. The inverse algorithm invFFT-new has the same cost.

Remark 4. There are other possible decompositions of a polynomial c into two self-adjoint polynomials; for instance, one may find self-adjoint polynomials a and b such that $c = a + b/(1 + X)$, i.e. c is the sum of a self-adjoint and an X-adjoint polynomials. We here favour the $c = a + X^{n/2}b$ decomposition because it yields the same FFT representation as FFT-classic, thus keeping other operations in Falcon unchanged.

3.4 Input Range and Benchmarks

We focus here on FFT-new, with an integral input polynomial whose coefficients are represented over 32-bit integers. The computations which are performed over plain integers are correct only as long as they do not incur an overflow. In the opening loop of FFT-new (algorithm 8, steps 1-5), each value $c[j]$ is replaced with the sum or the difference of two coefficients of the input. The resulting values are then passed to FFT-selfadj-new (algorithm 6, steps 1-7) which runs $\ell - 2$ iterations of a loop; at each iteration, some of the values are replaced with the sum of two values produced by the previous iteration. In total, this means that $|c[j]| \leq 2^{\ell-1} \|c\|_\infty$ for any j, when reaching step 8 of FFT-selfadj-new; afterwards, values are converted to floating-point and no longer limited to the 32-bit integer range. Correspondingly, for degree $n \leq 1024$, if $\|c\|_\infty < 2^{22}$, then no overflow may occur in the entire FFT-new. All uses of FFT-new (and FFT-selfadj-new) in this paper will be shown to fulfill that condition.

We implemented and benchmarked these algorithms on an Arm Cortex M4 microcontroller (code is in C but the addition, addition-subtraction and multiplications are hand-optimized in assembly). For an integral polynomial held in 32-bit signed integers, FFT-classic runs in 611608 cycles (for degree $n = 512$), or 1369401 cycles (for $n = 1024$). With FFT-new, these figures drop to 385940 and 855782 cycles, respectively. We can see that the extra integer-only operations have a non-negligible cost, but the performance gain is nonetheless substantial.

3.5 Annex: Extension to NTT

The *Number Theoretic Transform* (NTT) is analogous to the FFT when working modulo a prime integer q. To apply the NTT on polynomials in $\mathbb{Z}_q[X]/\Phi_n$, we normally want $\Phi_n = X^n + 1$ to split over Z_q, i.e. that $q = 1 \bmod 2n$. This is a slightly restrictive condition. For instance, Falcon uses $q = 12289$ because it needs to work up to degree $n = 1024$, and 12289 is the smallest prime that is equal to 1 modulo 2048. It is possible to some extent to use a *partial* NTT if the modulus is not adequate for the full degree; for instance, ML-KEM[18] uses $q = 3329$, which is good for NTT up to degree 128, but applies it to polynomials modulo $X^{256}+1$. The method is simply to split the input a into a_{even} and a_{odd} and apply the NTT on both; however, multiplications of polynomials in such a partial NTT representation become more complicated:

$$(ab)(\zeta) = (a_{\text{even}}b_{\text{even}} + Xa_{\text{odd}}b_{\text{odd}})(\zeta^2) + \zeta(a_{\text{even}}b_{\text{odd}} + a_{\text{odd}}b_{\text{even}})(\zeta^2)$$

i.e. more operations in $\mathbb{Z}_q$ are needed compared to a simple coefficient-wise multiplication. Generally speaking, if we want to work with polynomials of degree $n = 2^\ell$ but the modulus allows the plain NTT only up to $2^{\ell-\gamma}$ then multiplications of two polynomials in partial NTT representation will need $O(2^\gamma n)$ multiplications in $\mathbb{Z}_q$. We thus prefer γ to remain small. For degree $n = 1024$, there are not many good choices of primes lower than 12289: 7681, 10753 and 11777 allow up to degree 256 (hence $2^\gamma = 4$), while 257, 769, 3329, 7937 and 9473 work natively only up to degree 128, and thus imply $2^\gamma = 8$ when $n = 1024$.

A different way is to consider $q = -1 \bmod n$. With such a modulus, -1 is not a square in $\mathbb{Z}_q$, and we can extend that field to $\mathbb{F}_{q^2}$ by adjoining a formal element i such that $i^2 = -1$; this is the same construction as complex numbers with regard to reals. Note that if $q = -1 \bmod n$ then there is some integer k such that $q = kn - 1$, which implies that $q^2 - 1 = (q-1)(q+1) = (kn-2)kn$. With $n = 2^\ell$ being even, we get that $q^2 = 1 \bmod 2n$: polynomial Φ_n splits over $\mathbb{F}_{q^2}$, and we can apply the FFT, including the improved FFT-new and invFFT-new algorithms. All operations on complex numbers, in particular conjugation, directly map to $\mathbb{F}_{q^2}$. We only need to arbitrarily choose a root δ of $\Phi_n = X^n + 1$ in $\mathbb{F}_{q^2}$, then use the roots $\zeta_{n,j} = \delta^{2j+1}$ for $j = 0$ to $n-1$. In total, we get an FFT and inverse FFT with good efficiency (comparable to a plain full-degree NTT), and multiplications of two polynomials in FFT representation (in $\mathbb{F}_{q^2}$) require $2n$ multiplications in $\mathbb{F}_q$ (the equivalent of $2^\gamma = 2$ in terms of efficiency). Available moduli less than 12289 and that support degree $n = 1024$ for this FFT in $\mathbb{F}_{q^2}$ are 5519, 6143 and 8191; this last value, in particular, is a Mersenne prime, for which modular reduction is easy, making it attractive from an implementation point of view. If we accept to use a partial FFT, which increases the cost of polynomial multiplication, we can use more moduli, e.g. 1279 (good up to degree 256 natively) or 127 (good up to degree 128).

Falcon itself is specified only with $q = 12289$, and the plain NTT is the way to go for that modulus. But the FFT in $\mathbb{F}_{q^2}$, and its improved implementation FFT-new, can be useful for designing new schemes (or new parameter sets for Falcon) since they offer reasonably fast operations over additional moduli that help strike a better trade-off between object sizes (e.g. encoded public key) and security.

4 Falcon Tree Optimization

While we primarily target RAM optimizations, we are also happy to reduce runtime cost. Floating-point values are both large (8 bytes each) and expensive, thus we would like to have fewer of them. The *Falcon tree* is a structure that depends only on the private key; a speed-optimized implementation for an application where it is expected that many messages will be signed with the same key (e.g. a TLS server) will usually precompute the tree when the private key is loaded; in that case, the time taken to build this tree does not matter much (since the cost is amortized over many signatures), but the storage size could be important. On the other hand, a RAM-optimized implementation, in particular in the context of small embedded systems (for which RAM is usually a very scarce resource), will usually resort to dynamically generating the Falcon tree along with each signature, keeping in RAM at any point only the relevant parts of the tree (i.e. the path from the root to the particular leaf which is currently used). In this section, we present some optimization techniques that help in both cases.

4.1 *1/X*-Adjoint Polynomials

The FalconTreeTop algorithm (algorithm 1) generates the root node of the Falcon tree, then uses FalconTreeInner (algorithm 2) to build the left and right sub-trees. By construction, the parameter to FalconTreeInner is a self-adjoint polynomial a, which the algorithm splits: $(b, c) = \mathsf{split}(a)$. As was previously noticed (see remark 2), splitting a self-adjoint polynomial yields a self-adjoint and an X-adjoint polynomials, i.e. $b^* = b$ and $c^* = Xc$. Polynomial c^* is itself $1/X$-adjoint, since $c^{**} = c = (1/X)Xc = (1/X)c^*$. The root of the sub-tree returned by FalconTreeInner is the value $L = c^*/b$, which is then an $1/X$-adjoint polynomial.

Previous implementations of Falcon stored L as a generic polynomial in FFT representation, using $m/2$ floating-point elements ($4m$ bytes) for an input polynomial $a \in \mathbb{R}[X]/\Phi_m$ (the split operation then implies that b and c, and thus L, have degree $m/2$). However, we can instead store $L/(1+X)$, which is self-adjoint and requires only $m/4$ floating-point elements ($2m$ bytes). Algorithm LDL-new (algorithm 10) takes as input a (self-adjoint, FFT representation), splits it into b (i.e. a_{even}) and c (i.e. a_{odd}), then returns polynomials d and λ (both self-adjoint and in FFT representation) such that:

$$d = b - cc^*/b$$
$$\lambda = L/(1+X)$$

for $L = c^*/b$. This is mostly what FalconTreeInner computes, except for the specific representation of L as the self-adjoint polynomial λ.

In LDL-new, steps 2-4 represent $(b, c) = \mathsf{split}(a)$, with c being represented by $c' = (1+X)c$; then, variables u and v are equal to $b[j]$ and $c'[j]$, respectively (b and c' are self-adjoint). Then, we want:

$$\lambda = \frac{c^*}{(1+X)b} = \frac{c'}{(1+X)^*(1+X)b} = \left(\frac{X}{(1+X)^2}\right)\frac{c'}{b}$$

Hence, the constant $C[m/4+j]$ is equal to $(1+X)^2/X$ evaluated over the roots ζ of $\Phi_{m/2}$ (equivalenty, the roots ζ^2 of Φ_m) in the correct bit-reversal order, namely:

$$C[2^{\alpha-1} + \mathsf{rev}(\alpha-1, j)] = 2 + 2\cos((4j+1)\pi/2^\alpha)$$

Algorithm 10 LDL-new

Input: $a \in \mathbb{R}[X]/\Phi_m$ such that $a = a^*$ (FFT, half-size)
Output: $b, d = b - cc^*/b$ and $\lambda = c^*/(b(1+X))$ for $(b, c) = \mathsf{split}(a)$ (FFT, degree $m/2$, half-size)

1: **for** $j = 0$ **to** $m/4 - 1$ **do**
2: $(x, y) \leftarrow (a[2j], a[2j+1])$
3: $u \leftarrow (x + y)/2$
4: $v \leftarrow (x - y)R'[m/4 + j]$ ▷ Same constants $R'[j]$ as in invFFT-selfadj-new.
5: $\lambda[j] \leftarrow v/(uC[m/4 + j])$
6: $d[j] \leftarrow u - v\lambda[j]$

for $j = 0$ to $2^{\alpha-1}$. To support Falcon up to degree $n = 2^\ell$, these constants must be precomputed for α up to $\ell - 1$ (since the largest d and λ are for degree $n/2$), for a total size of 4096 bytes for $n = 1024$ and using 64-bit floating-point values.

When using the Falcon tree in ffSampling, the tree node value L must be multiplied with a generic polynomial, both being in FFT representation (algorithm 3, step 10). Since we do not have L, but $\lambda = L/(1 + X)$, this multiplication is modified; algorithm mul-iXadj (algorithm 11) performs this operation.

Algorithm 11 mul-iXadj

Input: $a \in \mathbb{R}[X]/\Phi_m$ (FFT), $\lambda \in \mathbb{R}[X]/\Phi_m$ such that $\lambda^* = \lambda$ (FFT, half-size)
Output: $d = (1 + X)\lambda a$ (FFT)

1: **for** $j = 0$ **to** $n/2 - 1$ **do**
2: $(x_r, x_i) \leftarrow (a[j], a[j + n/2])$
3: $y \leftarrow \lambda[j]$
4: $\theta = yC[n/2 + j]/2$ ▷ Same constants $C[j]$ as in LDL-new
5: $\delta = D[n/2 + j]$
6: $d[j] \leftarrow \theta(x_r + x_i\delta)$
7: $d[j + n/2] \leftarrow \theta(x_i - x_r\delta)$

This uses the constants $D[j]$ which are equal to $i(\zeta - 1)/(\zeta + 1)$ for the roots ζ of Φ_m; in the appropriate bit-reversal order:

$$D[2^{\alpha-1} + \mathsf{rev}(\alpha - 1, j)] = \frac{-\sin((4j+1)\pi/2^\alpha)}{1 + \cos((4j+1)\pi/2^\alpha)}$$

Again, the $D[]$ array size is 4096 bytes when supporting Falcon up to $n = 1024$ and using 64-bit floating-point values.

4.2 Symplecticity

Symplecticity is a geometric property fulfilled by Falcon trees[25]. It provides some useful relations between the left and right sub-trees of the root nodes; namely, the right sub-tree can be obtained from the left sub-tree by doing the following:

– Negate all values (L) of intermediate nodes.

– Exchange left and right children of each node.
– Replace each leaf value x with q^2/x.

If the Falcon tree is precomputed and stored in RAM, then these properties reduce both the storage size and the computational cost of building the tree; values of inner nodes of the right sub-tree need not be stored since they can be obtained by using the mirror nodes in the left sub-tree, with a very inexpensive negation, and the $n/2$ leaves of the right sub-tree are computed directly from the $n/2$ leaves of the left sub-tree.

The storage cost of a full Falcon tree is then:

– Top-level T.value $= (Ff^* + Gg^*)/(ff^* + gg^*)$: n floating-point values.
– Left sub-tree: $\ell - 1$ layers, each containing $2^{\ell-1-\alpha}$ values $\lambda \in \mathbb{R}[X]/\Phi_{2^\alpha}$ which are self-adjoint, for a total of $n/4$ floating-point values, and $n/2$ leaves.
– Right sub-tree: $n/2$ leaves (the inner nodes are omitted).

This means a total of $n(\ell + 7)/4$ floating-point values. If the values are 64-bit objects, then this means 16384 bytes (for $n = 512$) or 34816 bytes (for $n = 1024$).

In our implementation, we try to achieve low RAM usage, and thus we do not precompute the Falcon tree; instead, we compute it dynamically, keeping only a single path in RAM at a time, which implies that we have to recompute the L values in the right sub-tree. However, we can still leverage a consequence of symplecticity: the source value for the right sub-tree is the polynomial denoted d in FalconTreeTop (algorithm 1, step 8); this value is such that:

$$\begin{aligned}(ff^* + gg^*)d &= (FF^* + GG^*)(ff^* + gg^*) - (Ff^* + Gg^*)(F^*f + G^*g) \\ &= gF(gF)^* + fG(fG)^* - gF(fG)^* - fG(gF)^* \\ &= (gF - fG)(gF - fG)^* \\ &= q^2\end{aligned}$$

by application of the NTRU equation. We can thus obtain the source value for the right sub-tree as $d = q^2/(ff^* + gg^*) = q^2/a$, with a being the source value for the left sub-tree. Since a is in FFT representation and is self-adjoint, we get d with $n/2$ divisions on floating-point values.

An alternative way would be to specialize the left and right FalconTreeInner so that the dynamic reconstruction of the right sub-tree runs, in fact, the left sub-tree building and applies the symplectic negations and swaps, and the final divisions on the leaves. However, that would complicate the implementation (inner ffSampling calls would have to know whether they are exploring the left or right sub-tree); computing $d = q^2/a$ is simpler. Note that both methods involve the same number ($n/2$) of floating-point divisions.

4.3 Top-Level Polynomials

FalconTreeTop (algorithm 1) computes the top-level tree value L, and invokes FalconTreeInner on two polynomials equal to $a = ff^* + gg^*$ and, as was pointed out above, $d = q^2/a$. The top-level tree value is $L = b/a$. We therefore need to compute the two polynomials a and b. Note that both polynomials are integral; we can thus evaluate them over integers, and more precisely we compute a and b modulo $q = 12289$ and modulo $q' = 18433$, then assemble the resulting coefficients with the CRT to yield the result. This process works as long as the

integer coefficients can be shown to be less than $qq'/2$ in absolute value. In this subsection, we establish the maximum range of coefficients of a and b. We rely on two properties of valid Falcon private keys:

$$\|(F, G)\|_\infty \leq 2^7$$
$$\left\|(g, -f)\right\| \leq 1.17\sqrt{q}$$

The second property can also be expressed as follows:

$$\sum_{j=0}^{n-1} f[j]^2 + \sum_{j=0}^{n-1} g[j]^2 \leq 16822$$

Let's first consider $b = Ff^* + Gg^*$. For the purpose of range analysis, we can consider that all coefficients of F and G are equal to the upper bound of $\|(F, G)\|_\infty$, which is 2^7; hence:

$$\left\|Ff^* + Gg^*\right\|_\infty \leq 2^7 \sum_{j=0}^{n-1} (|f[j]| + |g[j]|)$$

We thus want to upper bound the sum of the (absolute values) of coefficients of f and g, under the condition that $\left\|(g, -f)\right\|^2 \leq 16822$. The following lemma will be useful:

Lemma 1. *Let $m \in \mathbb{N}$. Let $S_m = \{(x, y) \in \mathbb{N}^2 \mid x^2 + y^2 \leq m\}$. Then the maximum value of $x + y$ for $(x, y) \in S_m$ is achieved for some integers x and y such that $|x - y| \leq 1$.*

Proof. Let $(x, y) \in S_m$ that maximizes $x+y$. If $y \geq x+2$, then we define $(x', y') = (x+1, y-1)$. It holds that $x'^2 + y'^2 = x^2 + y^2 + 2(x - y) + 2 < x^2 + y^2$, therefore $(x', y') \in S_m$. However, $x' + y' = x + y$, thus (x', y') is also a maximizing pair. Applying this process repeatedly, we can find a pair $(x'', y'') \in S_m$ such that $|x'' - y''| \leq 1$ that also maximizes the sum $x'' + y''$. □

For any pair (f, g) of polynomials that fulfills the condition $\left\|(g, -f)\right\|^2 \leq 16822$, we can consider the smallest and largest of the $2n$ coefficients (in absolute value); if the difference between the two is 2 or more, then we can apply lemma 1 and obtain another pair (f', g') which is almost identical to (f, g) except that the two aforementioned coefficients have been replaced with values (x, y) such that $|x-y| \leq 1$, without changing the total sum $\sum_{j=0}^{n-1}(|f[j]| + |g[j]|)$, and still complying with the condition on $\left\|(g, -f)\right\|$. Doing so repeatedly shows that under that condition, $\sum_{j=0}^{n-1}(|f[j]| + |g[j]|)$ can be maximized when all coefficients of f and g are equal to either x or $x + 1$ for some integer x.

We can thus write that:

$$\left\|Ff^* + Gg^*\right\|_\infty \leq 2^7((2n - m)x + m(x + 1))$$

for some integers $m \in [0, 2n-1]$ and x which are such that $(2n-m)x^2+m(x+1)^2 \leq 16822$. Note that $(2n - m)x + m(x + 1) = 2nx + m$, which is maximized for any given x when m is maximized, under the constraint that $(2n - m)x^2 + m(x + 1)^2 \leq 16822$, which means that we need $m \leq (16822 - 2nx^2)/(2x + 1)$. Trying increasing values of x until there is no matching m, we can easily enumerate the largest possible value for $(2n - m)x + m(x + 1)$,

which is 5822 for $n = 1024$, and 4144 for $n = 512$ (this upper bound is smaller for lower degrees). Correspondibly, this demonstrates that:

$$\left\|Ff^* + Gg^*\right\|_\infty \leq 5822 \cdot 2^7 = 745216$$

which is both lower than $qq'/2$ (hence we can find the correct value of $Ff^* + Gg^*$ by working only modulo q and modulo q'), and lower than 2^{22}, which implies that $Ff^* + Gg^*$ is an adequate input to algorithm FFT-new with 32-bit integers (see section 3.4).

For $a = ff^* + gg^*$, we can prove a tighter range, using the following lemma:

Lemma 2. *For any polynomial $f \in \mathbb{R}[X]/\Phi_n$, $\left\|ff^*\right\|_\infty = \left\|f\right\|^2$.*

Proof. Let $k \in [1, n-1]$. A straightforward computation of ff^* shows that $ff^*[k]$ is the sum of values $\pm f[j]f^*[k-j]$, with the convention that $f[j-n] = -f[j]$ (which corresponds to reduction modulo $X^n + 1$). By definition of the adjoint, $|f^*[k-j]| = |f[j-k]|$. By triangular inequality, we have:

$$|ff^*[k]| \leq \sum_{j=0}^{n-1} |f[j]||f[j-k]|$$

For any reals x and y, we have $x^2 + y^2 - 2xy = (x-y)^2 \geq 0$, thus $xy \leq (x^2+y^2)/2$. Applying this rule to the sum above, we get:

$$|ff^*[k]| \leq \sum_{j=0}^{n-1} \frac{f[j]^2 + f[j-k]^2}{2}$$

Each $f[j]^2$ appears exactly twice in the right-hand side sum, hence:

$$|ff^*[k]| \leq \sum_{j=0}^{n-1} f[j]^2 = \left\|f\right\|^2$$

for all $k \in [1, n-1]$. At index 0, the inequality becomes an equality:

$$ff^*[0] = f[0]^2 - \sum_{j=1}^{n-1} f[j]f^*[n-j] = f[0]^2 + \sum_{j=1}^{n-1} f[j]^2 = \left\|f\right\|^2$$

which completes the proof. □

From lemma 2, we get that:

$$\left\|ff^* + gg^*\right\|_\infty \leq \left\|ff^*\right\|_\infty + \left\|gg^*\right\|_\infty = \left\|f\right\|^2 + \left\|g\right\|^2 = \left\|(g, -f)\right\|^2$$

Thus, all coefficients of $ff^* + gg^*$ are at most 16822 (in absolute value).

If we had $q' > 2 \cdot 16822$, then we could compute $ff^* + gg^*$ modulo q' only; it would be sufficient. However, that would require $q' > 33644$ (next prime appropriate for the NTT at $n = 1024$ is 40961), and there are computational advantages to sticking to a modulus lower than $2^{15} = 32768$ (in particular, this makes it easier to use some SIMD instructions of the Arm Cortex M4 such as `smulbb` which work only with *signed* 16-bit operands).

We will therefore use $q' = 18433$ and compute $ff^* + gg^*$ modulo q and modulo q'. The reconstruction of each coefficient as a plain integer is easier than with the CRT, though. Suppose that for a given index j, we obtain the coefficient modulo q as a value that normalizes as $x \in [-q/2, +q/2]$, and similarly the same coefficient modulo q' normalizes as $y \in [-q'/2, +q'/2]$. If $x = y$, $x = y + q$ or $x = y - q$, then x agrees with y, which is the value of the coefficient as an integer. Otherwise, the value y must be adjusted: if $y < 0$, then the correct value is $y + q'$, otherwise the correct value is $y - q'$. These operations are easy and inexpensive to apply. Moreover, if we have to compute $ff^* + gg^*$ twice (we will see that this is the case in our implementation, for RAM-saving reasons), then we can remember whether an adjustment to the value of the coefficient modulo q' was needed, and the second time we only compute the polynomial modulo q' and apply the adjustement based on the sign of each value. The storage space for such remembering is small (n bits, i.e. $n/8$ bytes); in our implementation, we store that bitfield temporarily in the output buffer for the signature.

5 Signature Generation Optimization

5.1 Optimization Techniques

We here present the main techniques we use to achieve a small RAM usage, as well as better performance on Arm Cortex M4 CPUs. The following ideas are used:

- Use FFT-new instead of FFT-classic.
- Perform computations on integer polynomials using modular integers, working modulo $q = 12289$ and $q' = 18433$.
- Build the Falcon tree dynamically, working only with self-adjoint polynomials (which use only half the storage space), as described in the previous section.
- Use only the fractional part of the input to ffSampling. Moreover, that input can be injected later in the process, at the leaf level in the Falcon tree rather than at the start, which avoids some FFTs.
- Record sampled integers as returned by SamplerZ and use these values to compute the signature, rather than the final polynomial returned by ffSampling. This removes the need to run an inverse FFT. In fact, invFFT-new is not used at all in our implementation.

Fractional Input. A well-known property of ffSampling is that it preserves the integral part of its input ([1], lemma 9): if we consider a Falcon tree T, a given state for the pseudorandom generator used in SamplerZ, and an input $\mathbf{t} = (t_0, t_1)$, yielding $\mathbf{z} = \text{ffSampling}(\mathbf{t}, \mathsf{T})$, then for any $\mathbf{k} = (k_0, k_1) \in (\mathbb{Z}[X]/\Phi_n)^2$, $\text{ffSampling}(\mathbf{t} + \mathbf{k}, \mathsf{T})$ should return $\mathbf{z} + \mathbf{k}$ with the same $\mathbf{z}$ if the same pseudorandom generator state is used. We can thus split the input into an integral and a fractional parts, perform the sampling on the fractional part only, then add back the integral part afterwards. In Falcon signature generation, the input vector is $(t_0, t_1) = (-cF/q, cf/q)$; we can split $cf/q = (cf \bmod q)/q + (cf - (cf \bmod q))/q$, the second part being integral. We will thus run ffSampling on $((-cF \bmod q)/q, (cf \bmod q)/q)$.

Late Addition. Algorithm ffSampling only repeatedly splits its input, and splitting (in coefficient representation) is a data routing operation, i.e. it does not actually modify the coefficient values. In algorithm 3 (step 10), the polynomial t'_0 is obtained by *adding* $(t_1 - z_1)L$ (produced by the previous sampling steps) to the input t_0; addition commutes with splitting, thus we can delay this addition until it is really needed, which is at the leaf level. In other words, instead of computing (t_0, t_1) and passing it as parameter to ffSampling, we can run ffSampling on $(0, 0)$ and add coefficients of $((-cF \bmod q)/q, (cf \bmod q)/q)$ only at the point where SamplerZ is called (algorithm 3, steps 3 and 4). We keep $-cF \bmod q$ and $cf \bmod q$ as arrays of 16-bit signed values ($2n$ bytes each); as is usual in splitting-based algorithms, these coefficients are consumed in bit-reversal order. The division by q, and conversion to floating-point, is only done at that point. This late addition has the benefit of avoiding an FFT computation: the FFT representation of 0 is 0. We can also skip some of the computations in ffSampling by propagating a flag that indicates that the input value is statically known to be 0 at this point. Apart from the obvious speed benefits (not running the FFT is always faster than running it), this also saves some RAM space because applying the FFT on a degree-n polynomial returns it as $8n$ bytes (with 64-bit floating-point elements), while working modulo q only needs $2n$ bytes.

Signature Building. The Gaussian sampler SamplerZ produces integer values (algorithm 3, steps 3 and 4); these values are ultimately assembled into the vector $\mathbf{z} = (z_0, z_1)$ and the overall ffSampling algorithm returns $\mathbf{z}$, normally in FFT representation, which the caller subtracts from the input $\mathbf{t}$. Using this vector entails performing an inverse FFT at some point. However, we can also discard it entirely; instead, we can record the coefficients of the sampled z_0 and z_1 in an alternate buffer (for instance, the same buffer that provides the coefficients of $-cF \bmod q$ and $cf \bmod q$ can be used to receive the sampled z_0 and z_1). The signature value is ultimately computed as:

$$\begin{aligned}\mathbf{s} &= (\mathbf{t} - \mathbf{z})\mathbf{B} \\ &= \mathbf{t}\mathbf{B} - \mathbf{z}\mathbf{B} \\ &= (c - gz_0 - Gz_1, fz_0 + Fz_1)\end{aligned}$$

These polynomials are all integral, hence we can compute the signature by working modulo q or modulo q'.

Merged Tree Building and Sampling. Using the techniques above, we merge the dynamic Falcon tree building and the Fourier sampling into algorithm ffTreeSampling (algorithm 12). It takes as input polynomials t and a, both from $\mathbb{R}[X]/\Phi_m$; a is self-adjoint and represents the input to FalconTreeInner, while t is the input to ffSampling (before splitting).

Algorithm 12 ffTreeSampling

Input: $t \in \mathbb{R}[X]/\Phi_m$ (FFT), $a \in \mathbb{R}[X]/\Phi_m$ (FFT, self-adjoint, half-size), I/O state W
Output: $u = t - z \in \mathbb{R}[X]/\Phi_m$ (FFT)

if $m = 2$ **then**
  $\sigma' \leftarrow \sigma_n / \sqrt{a[0]}$ ▷ Leaf normalization step.
  $(e_0, e_1) \leftarrow$ next-input(W)
  $z_0 \leftarrow$ SamplerZ($t[0] + e_0$) ▷ Late addition of input coefficient.
  $u[0] \leftarrow t[0] - z_0$
  $z_1 \leftarrow$ SamplerZ($t[1] + e_1$) ▷ Late addition of input coefficient.
  $u[1] \leftarrow t[1] - z_1$
  push-sample(W, z_0, z_1) ▷ Sampled integers are saved.
else
  $(t_0, t_1) \leftarrow$ split(t)
  $(b, d, \lambda) \leftarrow$ LDL-new(a)
  $u_1 \leftarrow$ ffTreeSampling(t_1, d, W) ▷ $u_1 = t_1 - z_1$
  $v_1 \leftarrow$ mul-iXadj(u_1, λ) ▷ $v_1 = (t_1 - z_1)L$
  $t'_0 \leftarrow t_0 + v_1$ ▷ $v_1 = t'_0 - t_0$
  $v_0 \leftarrow$ ffTreeSampling(t'_0, b, W) ▷ $v_0 = t'_0 - z_0$
  $u_0 \leftarrow v_0 - v_1$ ▷ $u_0 = t_0 - z_0$
  $u \leftarrow$ merge(u_0, u_1) ▷ $u = t - z$
return u

The running state W encapsulates an input/output buffer w for 16-bit values, and a counter k. Initially, the buffer contains the coefficient of the fractional additional input (e.g.

$cf \bmod q$) and the counter is set to $n/2 = 2^{\ell-1}$; each invocation of next-input (algorithm 13) returns two values from the buffer, while each invocation of push-sample (algorithm 14) receives two sampled integers that replace the two previously returned values in the buffer.

Algorithm 13 next-input

Input: $W = (w, k)$ (buffer w, counter k)
Output: (e_0, e_1); k is updated
1: $k \leftarrow k - 1$
2: $j \leftarrow \mathsf{rev}(\ell - 1, k)$
3: **return** $(w[j]/q, w[j + n/2]/q)$

Algorithm 14 push-sample

Input: $W = (w, k)$ (buffer w, counter k), integers (z_0, z_1)
Output: w is updated
1: $j \leftarrow \mathsf{rev}(\ell - 1, k)$
2: $w[j] \leftarrow z_0$
3: $w[j + n/2] \leftarrow z_1$

5.2 Full Signing Procedure

Putting these techniques together, we obtain the following signature generation process:

1. Compute the message representative μ, generate the random seed r, and sample the polynomial $c \in \mathbb{Z}_q[X]/\Phi_n$.
2. Compute $a = ff^* + gg^*$ by working modulo q and modulo q', and rebuild its plain integer coefficients (as shown in section 4.3, these coefficients are in $[-16822, +16822]$ and thus only need 2 bytes each).
3. Convert a to FFT with FFT-selfadj-new, then compute $d = q^2/a$ (coefficient-wise floating-point division, since we operate on the FFT representation).
4. Compute $w = cf \bmod q$, normalized to coefficients in $[-q/2, +q/2]$, and set counter $k = n/2$. w and k are the two components of state W.
5. Run ffTreeSampling$(0, d, W)$. This yields $u_1 = t_1 - z_1$. Note that we provided only a fractional input ($(cf \bmod q)/q$, not the full cf/q) but the missing integral parts cancel out in u_1 (we virtually subtract the same integral polynomial from both t_1 and z_1).
6. Save z_1 (currently stored in w) into another buffer.
7. Compute G from the NTRU equation ($G = gF/f \bmod q$) then $b = Ff^* + Gg^*$ (modulo q and modulo q', then reassembled as plain 32-bit integers), which is then converted to FFT representation and multiplied with $t_1 - z_1$.
8. Recompute $a = ff^* + gg^*$, first as integers, then converted to FFT. This value was already computed previously but we did not keep it for RAM-saving reasons. As pointed out

previously, this recomputation can be done modulo q' if we remember which coefficients modulo q' had to be adjusted to yield the plain integer value.

9. Divide the value $(t_1 - z_1)(Ff^* + Gg^*)$ by $ff^* + gg^*$. Both polynomials are in FFT representation and $ff^* + gg^*$ is self-adjoint, so this uses only $n/2$ floating-point inversions. This yields $(t_1 - z_1)L$ (in FFT representation).
10. Set w to $-cF \bmod q$, normalized to coefficients in $[-q/2, +q/2]$, and reset k to value $n/2$.
11. Run ffTreeSampling$((t_1 - z_1)L, a, W)$. We discard the returned value; only z_0 (now in buffer w) will be used afterwards.
12. Build the second part of the signature $s_1 = fz_0 + Fz_1$. We do not actually have z_0 and z_1, since the sampling was performed over the fractional parts; we really have obtained z'_0 and z'_1 such that:

$$z_0 = \frac{-cF - (-cF \bmod q)}{q} + z'_0$$
$$z_1 = \frac{cf - (cf \bmod q)}{q} + z'_1$$

This alters the signature computation as follows:

$$\begin{aligned} s_1 &= \left(z'_0 + \frac{-cF}{q} - \frac{-cF \bmod q}{q}\right) f + \left(z'_1 + \frac{cf}{q} - \frac{cf \bmod q}{q}\right) F \\ &= fz'_0 + f\frac{(cF \bmod q)}{q} + Fz'_1 - F\frac{(cf \bmod q)}{q} \end{aligned}$$

Note that s_1 is integral, hence we can perform that computation modulo q'; the division by q is then a multiplication by the inverse of q modulo q'. Working modulo q' is sufficient since a valid signature is small ($\|s_1\|_\infty \leq 840$), hence there is no loss of information.

13. Once s_1 is recomputed, the signature verification process is performed; this is easier than recomputing $s_0 = c - gz_0 - Gz_1$, and it more obviously matches what verifiers will compute. The verification process needs the public polynomial h, which is recomputed as $h = g/f \bmod q$ at this point.
14. If the verification succeeds ($\|(s_0, s_1)\|$ and $\|(s_0, s_1)\|_\infty$ are low enough) and the signature s_1 is successfully encoded into bytes within the expected output signature size, then a success is reported. Otherwise, the whole process starts again (μ is reused, but a new seed r is generated, hence a new polynomial c).

The entire procedure can fit in a RAM buffer of size $20n$ bytes (hence 10 or 20 kilobytes, for $n = 512$ or $n = 1024$). In ffTreeSampling (algorithm 12), the input uses $12n$ bytes ($8n$ bytes for t, $4n$ bytes for a, which is self-adjoint); when doing the first recursive invocation (step 12), values t_0 ($4n$ bytes), b ($2n$ bytes) and λ ($2n$ bytes) must be retained, while for the second recursive invocation (step 15), values u_0 ($4n$ bytes) and v_1 ($4n$ bytes) must be retained. In both cases, the function must reserve $8n$ bytes before calling itself recursively at half-degree; this leads to a RAM usage of $16n$ bytes by ffTreeSampling. On top of that, we need two $2n$-byte buffers for providing the additional input ($-cF \bmod q$, $cf \bmod q$) and receiving z_0 and z_1, hence the total RAM usage of $20n$ bytes.

5.3 Precision

As pointed out before, both the Falcon tree building and the signature generation are formally expressed over polynomials in $\mathbb{Q}[X]/\Phi_n$, whose exact value can be computed, though this is expensive since the denominators are huge. Practical implementations of Falcon use approximations, with floating-point or fixed-point values. The Falcon security analysis shows that security is maintained as long as the approximations are "good enough", specifically in the inputs to the SamplerZ algorithm. Each SamplerZ invocation uses as parameters a centre x and a width σ', and returns an integer sampled along a Gaussian distribution centred on x with standard deviation σ'.

In order to estimate how precise our implementation is, we generated 100 signatures with random keys (for both $n = 512$ and $n = 1024$) using our new implementation. We also generated the same signatures with the "standard" implementation, using the same private keys and pseudorandom seeds, thus resulting in the exact same signature values. Finally, we computed the mathematically exact rational values for the inputs to SamplerZ (using a perfunctory Python/Sage implementation[4]), in order to measure how much the two implementations deviate from the "perfect" values. At degree n, $2n$ centres x and n widths σ' are computed:

- For $j = 0$ to $2n - 1$, if the measured centre is $x[j]$, and the mathematically exact centre is $x_e[j]$, then the deviation is $|x[j] - x_e[j]|$.
- For $j = 0$ to $n - 1$, if the measured width is $\sigma'[j]$, and the mathematically exact width is $\sigma'_e[j]$, then the deviation is $|\sigma'[j]/\sigma'_e[j] - 1|$.

Averages of the deviations, for each index j, over 100 signatures and with both the "standard" and the new implementations are shown on figures 1 (for $n = 512$) and 2 (for $n = 1024$). Average values are reported in base-2 logarithmic scale.

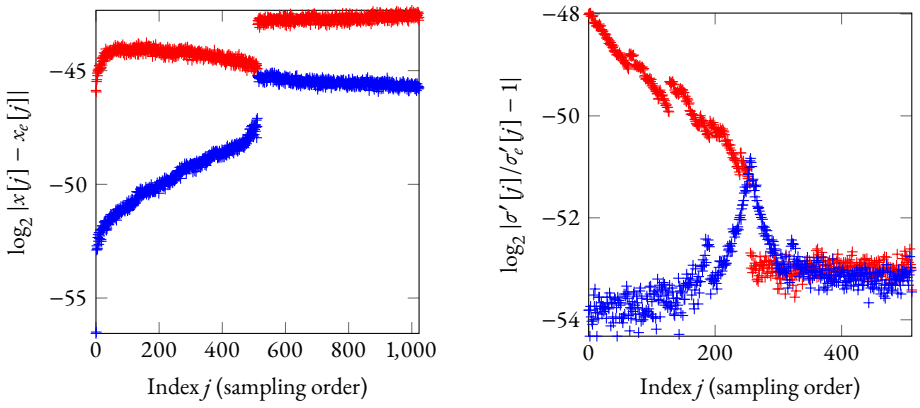

Fig. 1: Average deviations of centres x and widths σ' from exact values of the "standard" (red) and new (blue) implementations ($n = 512$).

[4] At $n = 1024$, this Sage implementation takes close to 5 minutes to compute one signature, which highlights how expensive the process is, and why using approximations is necessary.

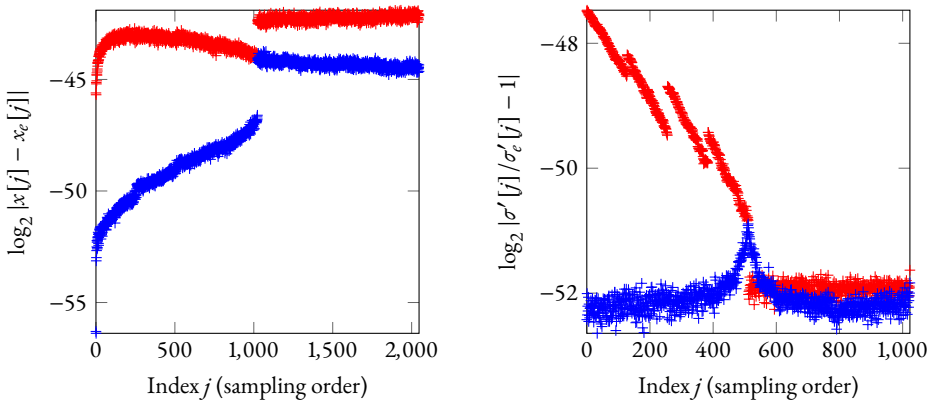

Fig. 2: Average deviations of centres x and widths σ' from exact values of the "standard" (red) and new (blue) implementations ($n = 1024$).

The main observation is that the new implementation consistently achieves better or at least as good precision as the "standard" code. The precision of the "standard" code is already deemed good enough for the Falcon scheme security goals, with deviations not being exploitable for attacks within the target security levels; in a sense, it is not possible to do better than that. However, we can still claim that the new implementation is also secure, and might have a bit more margin for alternate ways to compute values.

Why the new implementation seems to have better precision is not entirely clear, but it is probably due to a combination of these factors:

- Sampling over only the fractional domain implies that the centres provided to SamplerZ are lower (in absolute value), which, in floating-point formats, means that more bits are available for the fractional part.
- Computing the input for building the right Falcon sub-tree as $d = q^2/(ff^* + gg^*)$ entails use of smaller intermediate values than the "standard" method that uses $Ff^* + Gg^*$ and $FF^* + GG^*$, the latter having somewhat large coefficients. This, in particular, should explain why the "standard" implementation has relatively poor precision for the sampling widths in the first half of the process (when the right sub-tree is used).
- The late addition of inputs $(cf \bmod q)/q$ and $(-cF \bmod q)/q$ means that these values do not go through a FFT and inverse, and thus do not accumulate errors; at the point that they are added to the inputs of SamplerZ, they have the maximum possible precision that the floating-point format can support.
- Generally speaking, floating-point operations return approximate values, and errors tend to accumulate; the new implementation does fewer floating-point operations, thus entailing less error accumulation.

5.4 Reproducibility and Random Hedging

As figures 1 and 2 make explicit, our new implementation produces centres and widths that are not exactly the same values as the centres and widths generated by the "standard" imple-

mentation. In that case, it is unavoidable that even if using the same pseudorandom seeds for sampling, the two implementations may diverge: SamplerZ uses rejection sampling, which necessarily has threshold effects. If a sampled value is below the threshold for one implementation, but over the threshold for another implementation for whom the threshold is slightly different, then a divergence occurs.

To estimate how often divergences occur, we generated one million Falcon key pairs (at $n = 512$) and computed one signature with each key, with both the "standard" and the new implementations, using identical seeds. The generated signatures were identical in all cases except 124, which means that divergences occur at a rate of about 1/8000, which interestingly matches the rate observed by [1]. Within SamplerZ, the main threshold effect is in the split of the centre into integral and fractional parts: if the mathematically exact value is an integer, then each implementation will return an approximation which may be slightly above or below that integer, the two cases leading to diverging sampling processes. In the 124 observed divergences, 121 were in that case, all in either the first two or last two invocations of SamplerZ. 3 of the divergences, however, were for centres not particularly near an integer, and came from the rejecting sampling itself.

The combination of user-provided pseudorandom seeds and divergences that, while uncommon, still observationally happen, is dangerous: if two distinct implementations happen to use the same private key over the same message and with the same seed, and they hit a divergence, then the observation of the resulting signatures may reveal some information on the private key. To avoid this issue, our new implementation processes the user-provided seed in a different way. Namely, for a user-provided random seed ρ, an internal 40-byte seed is derived with value $H(\mathsf{id} \,\|\, H(f \,\|\, g) \,\|\, \mu \,\|\, \rho)$, with H being SHAKE256 (with a 40-byte output), f and g being injected as encoded in the private key, and id being a conventional fixed string[5]. In the "standard" implementation, id is the empty string; in the new implementation, it is an 8-byte fixed string. The use of such hashing, combining the input seed with the private key and the message representative, is an instance of "randomness hedging" and is considered good security engineering (it maintains security even if the source of randomness that produced the input seed is of poor quality). With a different label value in id, it also avoids the deleterious consequences of two implementations diverging on the same inputs.

Of course, this different key derivation prevents reproducibility of test vectors; all the experiments conducted above, to measure precision and reproducibility, were performed with id set to the same value (the empty string) as the "standard" implementation. In the published code, the new implementation has its own id and *none* of the test vectors from the "standard" code are reproduced. As explained above, this is needed for practical security if deployed applications use both the "standard" code and this new implementation with the same private keys, and inject fixed seeds (which is certainly not recommended, but may happen as long as the API that accepts an explicit seed exists).

5.5 Benchmarks

Within the course of writing our new implementation, we wrote some improved functions for computations in $\mathbb{Z}_q[X]/\Phi_n$, leveraging SSE2 (on x86) and NEON (on aarch64) SIMD instructions, and using Montgomery's trick for inverting polynomials in NTT representa-

[5] This process is in file `sign.c`, function `make_seedbuf()`.

tion with a single inversion in $\mathbb{Z}_q$. These optimizations were backported to the "standard" implementation, where they provide some relatively minor speed-ups.

All our code is constant-time, within the limitations of such exercises in the presence of modern compilers[22].

RAM usage. The "standard" code requires a temporary area of size $59n$ bytes; this is in addition to a stack space of slightly over 1 kB (stack usage depends on the used compiler). This translates to about 31 kB for $n = 512$, 61 kB for $n = 1024$. The new implementation significantly improves on that, since it reduces the temporary area size to $20n$ bytes; stack usage has also been slightly reduced (measured at 992 bytes on our Arm Cortex M4 test platform). In other words, we achieve a RAM usage of about **11 kB** (for $n = 512$) or **21 kB** (for $n = 1024$). Compared to the "standard" implementation, we divided RAM requirements by a factor of 3. This makes signature generation substantially more plausible on small embedded systems, including smartcards, were RAM resources are often very scarce. It may be noted that this RAM usage is now slightly below that of the key pair generation (at $22n$ bytes for the temporary area, and less than 1 kB of stack space).

Speed. The test x86 platform is an Intel Core i5-8259U (Coffee Lake, a Skylake variant) running at 2.3 GHz, in 64-bit mode. TurboBoost is disabled and the machine (running an up-to-date Ubuntu 24.04) is otherwise idle. The test aarch64 machine is a Raspberry Pi 5, with a Broadcom BCM2712 CPU (Arm Cortex-A76, 2.4 GHz); it also runs Ubuntu 24.04. For both machines, the C compiler is Clang 18.1.3, with optimization flags `-O2`. The C code on x86 leverages SSE2 but not AVX2 opcodes[6]. To represent embedded systems, we use an STM32F407VGT6 microcontroller (Arm Cortex M4F core); the code is uploaded in the SRAM block, while the data and stack are located in the CCM area: this specific setup allows more efficient loading of the code (with no Flash writes, thus avoiding wearing out the Flash chip) and, more importantly, running at 168 MHz with no wait states, which speeds up long-running benchmarks. The Arm Cortex M4F code is mostly in C (compiled with GCC 13.2.1) but with some routines in hand-optimized assembly, in particular the floating-point operations, as well as the NTT and inverse NTT modulo q and modulo q'. The recursive call in ffTreeSampling is also written in assembly in order to reduce the amount of stack space consumed in the recursion.

Table 1 shows the achieved speed, expressed in average clock cycle count for generating a signature, on the three platforms, with both standard degrees ($n = 512$ and $n = 1024$) and both implementations.

We can observe that the new code is slightly slower than the "standard" implementation on large CPUs with efficient hardware support for floating-point operations (cost is increased by 17% on x86, 27% on aarch64); on Arm Cortex M4, though, the reverse happens: the new code is faster (cost is reduced by 21%). The new code performs fewer floating-point opera-

[6] Some AVX2-specific code is used in key pair generation and in signature verification, but not in signature generation. Since the x86 ABI only guarantees presence of SSE2, use of AVX2 requires runtime testing of support, and code duplication, which is cumbersome from an engineering perspective. Right now, neither the "standard" code nor the new implementation use AVX2 for signature generation; using AVX2 would presumably speed things up for both, though it would not make the whole process twice faster, since at least SamplerZ is non-parallel.

Platform	Falcon-512		Falcon-1024	
	standard	new code	standard	new code
x86 (64-bit, SSE2)	872000	1023000	1759000	2064000
aarch64 (64-bit, NEON)	823000	1045000	1660000	2113000
Arm Cortex M4 (32-bit)	17020000	13450000	36570000	28580000

Table 1: Speed benchmarks of the "standard" and new implementations.

tions, but many more integer computations (modulo q and q'), so the overall performance depends on the relative cost of floating-point and integer operations on the involved hardware. In our new code, there are no fewer that 22 NTTs (12 modulo q, 10 modulo q') and 12 inverse NTTs (8 modulo q, 4 modulo q'); we also need to invert f modulo q twice, and the message representative is derived into polynomial c three times. These large numbers of operations are due to the optimization for space, that prevents us from caching intermediate values across calls to ffTreeSampling. On the other hand, there are only 3 FFTs, two of which being FFT-selfadj-new (for $ff^* + gg^*$, which is computed twice), and no inverse FFT at all.

On Arm Cortex M4, the expensive floating-point operations are additions, combined addition-subtractions, multiplications, divisions and square roots, all of which being implemented with hand-optimized assembly routines (constant-time). A signature generation at $n = 512$ involves on average 29416 additions, 13322 combined addition-subtractions, 48601 multiplications, 2817 divisions, and 512 square roots; for additions, addition-subtractions and multiplications, this is less than half the numbers on the "standard" code. In total, these floating-point operations account for about 6.6 million cycles, i.e. slightly less than half of the total cost (in the "standard" implementation, they make up to 66% of the cost). Presumably, a fixed-point implementation would reduce the cost of these operations, and correspondingly speed up the signature generation.

Comparison with ML-DSA. ML-DSA[19] is the recently NIST-standardized "generic" lattice-based post-quantum signature scheme. For signature generation, the "`m4f`" implementation from the pqm4 project[14] computes an ML-DSA-44 signature with an average cost of 3.943 million cycles on Arm Cortex M4, which is certainly faster than the 13.45 million cycles of our new Falcon-512 implementation. However, it does that with a much larger RAM space usage (about 49 kB, compared to 11 kB for our new implementation), which is likely to hinder deployment on smartcards and similarly small embedded systems. [6] presents a RAM-optimized ML-DSA implementation which can fit ML-DSA-44 signature generation in 5 kB, but the runtime cost raises to 18.47 million cycles.

An additional consideration is the variance on signature generation cost. Both Falcon and ML-DSA may fail to generate a signature at first, and then need to retry. For Falcon with the current behaviour (and especially the enforcement of $\|(s_0, s_1)\|_\infty \leq 840$), this happens with probability close to $1/2450$ at $n = 512$. This means that if a signature must be generated within a specific time frame with probability at least $1 - p$ for some small p, then the application must account for a number of successive attempts. If $p = 10^{-9}$ (for "nine nines" reliability) then the application must account for up to three signing iterations, or about 40 million cycles. For ML-DSA, the retry probability is much higher: on average, ML-

DSA-44 needs 4.25 iterations to succeed, and "nine nines" availability needs to account for no fewer than 78 iterations. The pqm4 benchmarks indicate that along with the average cost of 3.943 million cycles, the minimal observed cost is 1.813 million cycles, which would correspond to cases that succeed at first try. This allows us to estimate that each iteration has cost $(3.943 - 1.813)/3.25 = 0.655$ million cycles, in which case an ML-DSA instance that uses 78 iterations would use about 52 million cycles. This is more than our 40 million cycles, and this is with the "fast" ML-DSA implementation which uses 49 kB of RAM. In that sense, one may argue that with our implementation, Falcon-512 signature generation on Arm Cortex M4 is faster *and* smaller than ML-DSA-44. This makes Falcon quite competitive.

5.6 Future Work

The following points need further exploration:

- The fixed-point techniques from [1] should be merged into this new implementation, and benchmarked. This should yield some performance improvement on Arm Cortex M4, and, more importantly, it would presumably make it easier to mask against side-channel attacks.
- FFT-new replaces FFT-classic, but the split and merge operations from ffSampling are unchanged; these operations really are the inner primitives of FFT-classic and its inverse. There might be possible improvements to this process similar to the improvements in FFT-new.
- The overall code footprint is non-negligible, at about 53 kB for the signature generation code (including 13 kB which are also used by key pair generation and verification, in particular the SHAKE implementation, and the NTT tables modulo q). The floating-point code includes 20 kB of constants (R for FFT-selfadj-new, R', C and D for LDL-new and mul-iXadj, and $\sin(j\pi/1024)$ for $j = 0$ to 511 for split and merge). Embedded systems usually have more ROM/Flash than RAM[7], but this is still a resource that should be preserved, and some effort should be made at reducing code size.

[7] 1 bit of SRAM needs 6 transistors, but 1 bit of ROM or Flash uses about the space of a single transistor; moreover, SRAM continuously draws current to maintain its contents, proportionally to its size, but ROM and Flash do not.

References

1. D. De Almeida Braga, P.-A. Fouque, B. Lachguel and T. Prest, *Toward a Secure Fixed-Point Implementation of the Falcon Signature Scheme*, Advances in Cryptology - CRYPTO 2026 (part III), Lecture Notes in Computer Science, vol. 16802, pp. 166-197, 2026.
2. L. Babai, *On Lovasz' lattice reduction and the nearest lattice point problem*, Combinatorica, vol. 6, pp. 1-13, 1986.
3. P.-A. Berthet, *Masking, Sequences, and FALCON: A Theoretical Study on Masking Strategies Using Sequences for Non-Linear Operators in the FALCON Post-Quantum Signature*, https://eprint.iacr.org/2026/1534
4. P.-A. Berthet, J. Paillet, C. Tavernier, L. Bossuet and B. Colombier, *Masked Computation of the Floor Function and Its Application to the FALCON Signature*, IACR Communications in Cryptology, vol. 1, issue 4, 2025.
5. P.-A. Berthet, C. Tavernier, *Improving the masked division for the FALCON signature*, https://eprint.iacr.org/2025/628
6. J. Bos, J. Renes and A. Sprenkels, *Dilithium for Memory Constrained Devices*, Progress in Cryptology - AFRICACRYPT 2022, Lecture Notes in Computer Science, vol. 13503, pp. 217-235, 2022.
7. K.-Y. Chen and J.-P. Chen, *Masking Floating-Point Number Multiplication and Addition of Falcon*, IACR Transactions on Cryptographic Hardware and Embedded Systems, vol. 2024, issue 2, pp. 276-303, 2024.
8. S. Chen, J. Ming, Y. Liu, Y. Gao and Y. Zhou, *Masked Gadgets for Integer-Floating-Point Conversion with Applications to Falcon*, 2025 IEEE 43rd International Conference on Computer Design (ICCD), pp. 507-514, 2025.
9. J. Cooley and J. Tukey, *An algorithm for the machine calculation of complex Fourier series*, Mathematics of Computation, vol. 19, issue 90, pp. 297-301.
10. L. Ducas and T. Prest, *Fast Fourier Orthogonalization*, ISSAC '16: Proceedings of the 2016 ACM International Symposium on Symbolic and Algebraic Computation, pp. 191-198, 2016.
11. P.-A. Fouque, P. Gajland, H. de Groote, J. Janneck and E. Kiltz, *A Closer Look at Falcon*, Advances in Cryptology - EUROCRYPT 2026 (part IV), Lecture Notes in Computer Science, vol. 16544, pp. 3-31, 2026.
12. C. Gauss, *Nachlass: Theoria interpolationis methodo nova tractata*, Werke, vol. 3, pp. 265-303, Göttingen: Königliche Gesellschaft der Wissenschaften, 1866.
13. W. Gentleman and G. Sande, *Fast Fourier transforms – for fun and profit*, AFIPS '66 (Fall), pp. 563-578, 1966.
14. M. Kannwischer, J. Rijneveld, P. Schwabe and K. Stoffelen, *pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4*, https://eprint.iacr.org/2019/844
15. E. Karabulut and A. Aysu, *Masking FALCON's floating-point multiplication in hardware*, IACR Transactions on Cryptographic Hardware and Embedded Systems, vol. 2024, issue 4, pp. 483-508, 2024.
16. P. Klein, *Finding the closest lattice vector when it's unusually close*, SODA '00: Proceedings of the eleventh annual ACM-SIAM symposium on Discrete algorithms, pp. 937-941, 2000.
17. *Post-Quantum Cryptography*, National Institute of Standard and Technology, https://www.nist.gov/pqcrypto
18. Information Technology Laboratory, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, National Institute of Standard and Technology, FIPS 203, 2024.
19. Information Technology Laboratory, *Module-Lattice-Based Digital Signature Standard*, National Institute of Standard and Technology, FIPS 204, 2024.

20. T. Pornin and T. Prest, *More Efficient Algorithms for the NTRU Key Generation using the Field Norm*, Public Key Cryptography - PKC 2019 (part II), Lecture Notes in Computer Science, vol. 11443, pp. 504-533, 2019.
21. T. Pornin, *Improved Key Pair Generation for Falcon, BAT and Hawk*, https://eprint.iacr.org/2023/290
22. T. Pornin, *Constant-Time Code: The Pessimist Case*, https://eprint.iacr.org/2025/435
23. T. Pornin, *Improved (Again) Key Pair Generation for Falcon, BAT and Hawk*, https://eprint.iacr.org/2025/1239
24. T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte and Z. Zhang, *Falcon*, Technical report, National Institute of Standards and Technology, 2019, https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions
25. S. Sun, Y. Zhou, R. Zhang, Y. Tao, Z. Qiao and J. Ming, *Fast Fourier orthogonalization over NTRU lattices*, Information and Communications Security - ICICS 2022, Lecture Notes in Computer Science, vol. 13407, pp. 109-127, 2022.